

# 序知

A

I

个

人

体

系

化

知

识

库

软件工程文档合订本

制作说明：

项目名称：序知 AI 个人体系化知识库

课程：软件工程

指导老师：胡刚

日期：2026 年 6 月

# 序知 AI 个人体系化知识库 软件需求获取文档

## 目 录

|                   |    |
|-------------------|----|
| 1.项目定位.....       | 1  |
| 1.1 项目背景 .....    | 1  |
| 1.2 项目目的 .....    | 2  |
| 1.3 目标用户 .....    | 3  |
| 1.4 产品边界 .....    | 4  |
| 2.成员分工.....       | 5  |
| 3.软件需求.....       | 6  |
| 3.1 功能需求 .....    | 6  |
| 3.2 非功能需求 .....   | 8  |
| 3.3 数据需求 .....    | 9  |
| 4.软件需求描述分析.....   | 10 |
| 4.1 用户用例 .....    | 10 |
| 4.2 AI 服务用例 ..... | 22 |
| 4.3 数据库用例 .....   | 25 |

## 1.项目定位

### 1.1 项目背景

随着在线学习、远程办公、信息检索和大模型工具的普及，个人每天都会产生大量碎片化知识。课程笔记、会议纪要、读书摘录、网页资料、灵感想法、问题记录和项目复盘都可能具有长期价值，但这些内容在产生时通常是无序的、临时的、重复的，甚至存在语序颠倒和上下文缺失。

传统笔记软件主要解决“记录”和“存储”问题，用户需要自己建立目录、标签、分类和层级。实际使用过程中，很多用户会因为整理成本过高而只囤积素材，最后形成大量难以复用的资料堆。序知要解决的不是“写一篇好看的笔记”，而是解决“长期积累后如何自动形成个人知识体系”的问题。

本项目的核心定位是：用户自由无序录入，AI 全权体系化重构。用户不需要提前整理逻辑、排版或分类，只需要将原始素材放入系统；AI 负责从素材中理解含义、提取有效信息、建立主题、组织层级、生成知识节点并维护节点之间的关系。

### 1.2 项目目的

- 实现原始素材永久保留，保证用户输入内容不被 AI 覆盖或破坏。
- 实现 AI 单条解析，将杂乱素材转化为核心含义、关键词、主题和知识类型。
- 实现 AI 全库体系化重构，将分散素材整理为“一级主题—二级分支—知识节点”的结构。
- 实现来源引用，每个知识节点都能追溯到一个或多个原始素材。
- 实现知识网络维护，通过节点关系、版本历史和健康检查支撑长期演化。
- 实现知识复用，包括知识树浏览、个人知识库问答和 Markdown/JSON 导出。

### 1.3 目标用户

| 用户类型  | 需求特点                 | 典型使用场景                  |
|-------|----------------------|-------------------------|
| 学生    | 学习资料多、知识点散、复习时需要体系。  | 课程笔记、教材摘录、论文资料、错题和复习提纲。 |
| 职场用户  | 会议与项目资料频繁产生，需要形成经验库。 | 会议纪要、需求讨论、项目复盘、工作方法沉淀。  |
| 创作者   | 素材来源杂，灵感和参考资料需要专题聚合。 | 选题灵感、摘录、案例库、观点体系整理。     |
| 终身学习者 | 跨领域资料长期积累，需要自动关联。    | 读书笔记、公开课、网页剪藏、个人思考。     |

### 1.4 产品边界

- 序知不是普通笔记软件，不要求用户手动建立目录。
- 序知不是娱乐聊天工具，AI 输出必须服务个人知识体系化。
- 序知不做脱离用户知识库的外网编造，回答和重构应基于用户素材。

- 序知第一阶段不做多人协作、模板市场、公开分享和移动端 App。

## 2.成员分工

| 成员  | 学号          | 主要分工                                                     |
|-----|-------------|----------------------------------------------------------|
| 刘锦耀 | 20231060115 | 主要代码手。负责项目主框架、核心功能实现、AI 接口、前后端联调、GitHub 推送和 Vercel 部署验证。 |
| 马梓航 | 20231060267 | 负责需求获取、用户场景梳理、功能需求整理和用例规约协助。                             |
| 明曦  | 20231060270 | 负责界面流程梳理、用户使用说明、页面截图整理和交互体验检查。                           |
| 邓卓  | 20231060168 | 负责测试用例设计、测试执行记录、异常场景检查和测试报告整理。                           |
| 马敏楠 | 20231060189 | 负责数据库设计说明、接口文档整理、数据字典校对和报告格式检查。                          |

课程名称：软件工程。指导老师：胡刚。项目采用 AI 辅助软件开发方式完成，团队成员围绕需求、设计、编码、测试、部署和文档工作分工协作。

## 3.软件需求

### 3.1 功能需求

| 模块      | 核心功能                                                                                                                                       |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------|
| 用户认证    | <ul style="list-style-type: none"> <li>- 邮箱密码登录与注册；</li> <li>- 未登录用户跳转登录页；</li> <li>- 登录后读取当前用户会话。</li> </ul>                              |
| 素材入库    | <ul style="list-style-type: none"> <li>- 支持手动输入、粘贴、OCR、音频转写、PDF/Word 解析；</li> <li>- 保留 raw_content；</li> <li>- 支持状态筛选、搜索、编辑和删除。</li> </ul> |
| AI 单条解析 | <ul style="list-style-type: none"> <li>- 提取核心含义、有效信息、冗余信息、主题、知识类型、关键词和关联提示；</li> <li>- 输出结构化 JSON；</li> <li>- 失败时标记 failed。</li> </ul>     |
| 文本分块与向量 | <ul style="list-style-type: none"> <li>- 长素材切分为 chunks；</li> <li>- 生成 embedding；</li> <li>- 使用 pgvector 支持语义检索。</li> </ul>                 |

|          |                                                                                                                                  |
|----------|----------------------------------------------------------------------------------------------------------------------------------|
| AI 体系化重构 | <ul style="list-style-type: none"> <li>- 读取 analyzed 素材；</li> <li>- 生成一级主题、二级分支和知识节点；</li> <li>- 写入来源引用和版本快照。</li> </ul>         |
| 知识网络     | <ul style="list-style-type: none"> <li>- 维护 knowledge_edges；</li> <li>- 展示相关节点；</li> <li>- 支持后续扩展前置、支撑、矛盾、延伸等关系。</li> </ul>      |
| 知识问答     | <ul style="list-style-type: none"> <li>- 服务端基于当前用户知识库组织上下文；</li> <li>- 调用 AI 生成回答；</li> <li>- 未来支持问答回存。</li> </ul>               |
| 版本与健康    | <ul style="list-style-type: none"> <li>- 查看体系化版本；</li> <li>- 比较版本差异；</li> <li>- 检查重复、孤儿、无来源等知识库健康问题。</li> </ul>                  |
| 设置与导出    | <ul style="list-style-type: none"> <li>- 模型选择；</li> <li>- API Key 配置；</li> <li>- JSON 与 Markdown 导出；</li> <li>- 隐私说明。</li> </ul> |

### 3.2 非功能需求

- 性能需求：页面交互应保持流畅，体系化重构限制单次处理规模，避免上下文过长造成模型失败。
- 安全需求：所有业务表启用 RLS，服务端接口必须验证用户身份并按 user\_id 过滤。
- 可靠性需求：AI 调用失败不能影响原始素材，任务状态需要记录 completed 或 failed。
- 可维护性需求：核心业务类型统一定义，数据库迁移按模块拆分，API 路由职责清晰。
- 可扩展性需求：原始素材层、AI 理解层、知识网络层分离，便于后续支持增量更新。
- 可用性需求：工作台采用三栏结构，使用户能在一个界面完成输入、解析和浏览。

### 3.3 数据需求

| 数据对象 | 说明                    | 保存位置              |
|------|-----------------------|-------------------|
| 原始素材 | 用户输入或导入的原始内容，不做覆盖式改写。 | materials         |
| 解析结果 | AI 对单条素材的结构化理解。       | material_analysis |
| 素材分块 | 长文本切分后的检索单元和向量。       | material_chunks   |
| 知识主题 | AI 生成的一级主题和二级分支。      | knowledge_topics  |

|      |                      |                     |
|------|----------------------|---------------------|
| 知识节点 | 用户主要阅读和复用的体系化内容。     | knowledge_nodes     |
| 来源引用 | 节点与原始素材之间的证据关系。      | node_material_links |
| 知识关系 | 节点之间的相关、前置、支撑、矛盾等关系。 | knowledge_edges     |
| 版本快照 | 每次体系化结果的完整 JSON 快照。  | knowledge_versions  |

## 4.软件需求描述分析

### 4.1 用户用例

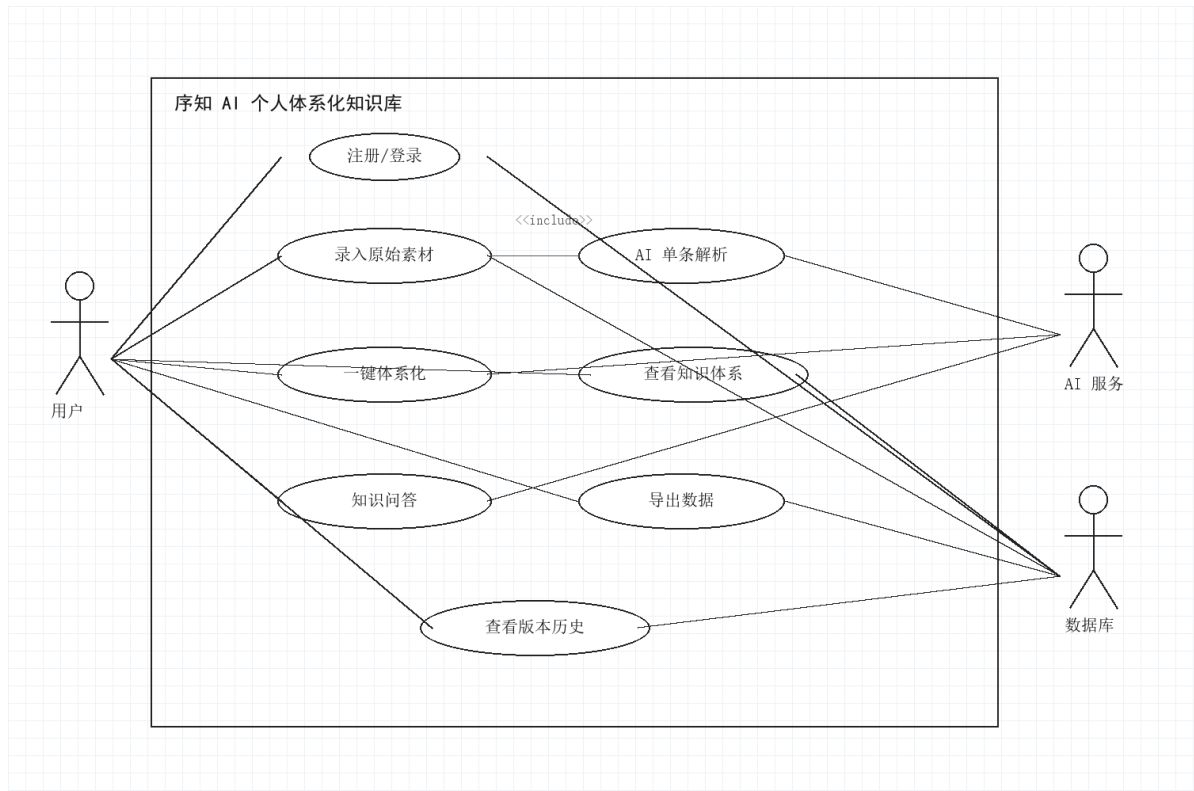


图 1 用户用例图

#### 4.1.1 用户登录

- 主要参与者：用户
- 系统职责：验证用户身份并创建会话
- 前置条件：用户拥有已注册邮箱和密码
- 后置条件：用户进入主页面
- 基本交互序列：
  - 1.用户打开登录页。
  - 2.用户输入邮箱和密码。

3. 用户点击登录按钮。
4. 系统调用 Supabase Auth 校验账号。
5. 系统创建会话并跳转工作台。
  - 扩展交互序列：
    1. 邮箱或密码错误时提示重新输入。
    2. 认证服务请求失败时在登录页展示错误。
  - 业务说明：登录是所有知识库操作的前提。

#### 4.1.2 新增原始素材

- 主要参与者：用户
- 系统职责：保存用户原始输入
- 前置条件：用户已登录
- 后置条件：materials 表新增 pending 记录
- 基本交互序列：
  1. 用户进入工作台。
  2. 用户输入标题和原始内容。
  3. 用户点击保存。
  4. 系统写入 raw\_content。
  5. 素材列表刷新。
- 扩展交互序列：
  1. 内容为空时阻止提交。
  2. 保存失败时提示错误。
- 业务说明：原始素材是事实源，后续 AI 只基于此进行解析和重构。



#### 4.1.3 编辑原始素材

- 主要参与者：用户
- 系统职责：更新素材标题和原文
- 前置条件：素材属于当前用户
- 后置条件：materials 更新 updated\_at
- 基本交互序列：
  1. 用户打开素材详情。
  2. 用户修改标题或内容。
  3. 用户点击保存。
  4. 系统更新数据库。
- 扩展交互序列：
  1. 无权限访问时返回 401 或 404。
  2. 内容过长时提示用户拆分。
- 业务说明：编辑后可重新解析，使知识体系基于最新素材。

#### 4.1.4 删除原始素材

- 主要参与者：用户
- 系统职责：删除素材及相关解析数据
- 前置条件：素材属于当前用户
- 后置条件：素材和级联数据被移除
- 基本交互序列：
  1. 用户选择素材。
  2. 用户点击删除。
  3. 系统确认操作。
  4. 系统删除记录。
- 扩展交互序列：
  1. 用户取消确认则不执行删除。
  2. 已被知识节点引用时提示可能影响体系。
- 业务说明：删除操作需要谨慎，因为来源引用会发生变化。

#### 4.1.5 AI 单条解析

- 主要参与者：用户、AI 服务
- 系统职责：调用 AI 输出结构化解析
- 前置条件：素材存在且未处于 analyzing
- 后置条件：保存 material\_analysis
- 基本交互序列：
  1. 用户点击解析。
  2. 系统将素材状态设为 analyzing。
  3. 系统调用 AI。
  4. AI 返回 JSON。
  5. 系统写入解析结果。
  6. 系统将状态设为 analyzed。
- 扩展交互序列：
  1. AI 失败时状态设为 failed。
  2. JSON 不合法时记录错误。
- 业务说明：解析层降低重构时噪声，提高体系化质量。

#### 4.1.6 文件导入解析

- 主要参与者：用户
- 系统职责：读取上传文档内容并转为素材
- 前置条件：用户选择 PDF 或 Word 文件
- 后置条件：文件文本进入素材层
- 基本交互序列：
  1. 用户选择文件。
  2. 系统调用 parse-file 接口。
  3. 系统抽取文本。
  4. 用户确认后保存为素材。
- 扩展交互序列：
  1. 文件格式不支持时提示错误。
  2. 解析内容为空时提示重新上传。
- 业务说明：该用例扩展了素材来源，但最终仍进入 materials。

#### 4.1.7 图片 OCR 入库

- 主要参与者：用户、AI 服务
- 系统职责：识别图片中的文字
- 前置条件：用户上传图片
- 后置条件：识别文本可保存为素材
- 基本交互序列：
  - 1.用户上传图片。
  - 2.系统调用 OCR 接口。
  - 3.AI 返回识别文本。
  - 4.用户确认内容。
  - 5.系统保存素材。
- 扩展交互序列：
  - 1.识别失败时提示重试。
  - 2.文本质量低时允许手动修正。
- 业务说明：OCR 能把截图、拍照笔记纳入知识库。

#### 4.1.8 音频转写入库

- 主要参与者：用户、AI 服务
- 系统职责：将录音转写为文本
- 前置条件：用户上传音频
- 后置条件：转写文本可保存为素材
- 基本交互序列：
  - 1.用户上传音频。
  - 2.系统调用转写接口。
  - 3.AI 返回转写文本。
  - 4.用户确认并保存。
- 扩展交互序列：
  - 1.音频过大时提示压缩。
  - 2.转写失败时保留错误信息。
- 业务说明：适合会议记录、课堂录音和临时口述灵感。

#### 4.1.9 一键体系化

- 主要参与者：用户、AI 服务、数据库
- 系统职责：生成知识体系并落库
- 前置条件：存在 analyzed 素材
- 后置条件：生成主题、节点、引用、关系和版本
- 基本交互序列：
  1. 用户点击一键体系化。
  2. 系统创建 reconstruction\_job。
  3. 系统读取已解析素材。
  4. AI 生成体系化 JSON。
  5. 系统写入知识网络。
  6. 系统保存版本快照。
- 扩展交互序列：
  1. 没有 analyzed 素材时提示无法体系化。
  2. AI 失败时任务标记 failed。
- 业务说明：这是序知区别于普通笔记软件的核心用例。

#### 4.1.10 查看知识体系

- 主要参与者：用户
- 系统职责：展示主题树和节点详情
- 前置条件：已存在知识节点
- 后置条件：用户看到体系化成果
- 基本交互序列：
  1. 用户进入知识页。
  2. 系统加载主题树。
  3. 用户展开主题。
  4. 用户点击节点。
  5. 系统显示节点内容和来源素材。
- 扩展交互序列：
  1. 无知识体系时提示先体系化。
  2. 节点被删除时刷新列表。
- 业务说明：该视图是用户使用 AI 重构结果的主要入口。

#### 4.1.11 查看相关节点

- 主要参与者：用户
- 系统职责：展示知识节点关系
- 前置条件：节点存在并有 edges
- 后置条件：用户理解知识间联系
- 基本交互序列：
  1. 用户打开节点详情。
  2. 系统查询 source/target 边。
  3. 系统展示相关节点。
- 扩展交互序列：
  1. 无相关节点时显示空状态。
- 业务说明：知识关系让系统从树状目录进一步升级为知识网络。

#### 4.1.12 知识问答

- 主要参与者：用户、AI 服务
- 系统职责：基于用户知识库回答问题
- 前置条件：用户已登录
- 后置条件：返回带知识库上下文的回答
- 基本交互序列：
  1. 用户输入问题。
  2. 系统检索素材和知识节点。
  3. 系统组织上下文。
  4. AI 生成回答。
  5. 前端显示回复。
- 扩展交互序列：
  1. 知识库为空时提示先添加素材。
  2. AI 失败时提示稍后重试。
- 业务说明：问答不是开放聊天，而是个人知识库的复用方式。

#### 4.1.13 查看版本历史

- 主要参与者：用户
- 系统职责：展示体系化历史快照
- 前置条件：用户至少执行过一次体系化
- 后置条件：用户可查看版本列表
- 基本交互序列：
  1. 用户打开历史记录。
  2. 系统读取 `knowledge_versions`。
  3. 用户选择版本。
  4. 系统显示摘要或差异。
- 扩展交互序列：
  1. 无历史版本时显示空状态。
- 业务说明：版本历史保证知识体系演化过程可追溯。

#### 4.1.14 导出个人数据

- 主要参与者：用户
- 系统职责：导出 JSON 或 Markdown
- 前置条件：用户已登录
- 后置条件：下载个人知识库备份
- 基本交互序列：
  1. 用户进入设置页。
  2. 用户点击导出。
  3. 系统读取用户数据。
  4. 系统生成文件并返回下载。
- 扩展交互序列：
  1. 导出失败时提示错误。
- 业务说明：导出能力保证数据可迁移，降低用户长期使用顾虑。

## 4.2 AI 服务用例

### 4.2.1 生成结构化解析 JSON

- 主要参与者：AI 服务
- 系统职责：按照系统 Prompt 输出固定字段
- 前置条件：收到素材文本和规则
- 后置条件：返回可落库 JSON
- 基本交互序列：
  1. 读取系统角色约束。
  2. 理解素材含义。
  3. 提取主题、关键词和知识类型。
  4. 返回 JSON。
- 扩展交互序列：
  1. 无法判断类型时选择 summary。
  2. 内容过短时仍输出最小结构。
- 业务说明：AI 服务不能脱离素材编造。

### 4.2.2 生成体系化知识树

- 主要参与者：AI 服务
- 系统职责：把多条素材重构为主题和节点
- 前置条件：收到已解析素材集合
- 后置条件：返回 system\_title、summary、topics
- 基本交互序列：
  1. 合并相似素材。
  2. 生成一级主题。
  3. 生成二级分支。
  4. 生成节点并回填 source\_material\_ids。
- 扩展交互序列：
  1. 素材过少时生成较小体系。
  2. 素材冲突时保留来源。
- 业务说明：体系化重构强调秩序，而不是扩写。

## 4.3 数据库用例

### 4.3.1 执行 RLS 权限隔离

- 主要参与者：数据库
- 系统职责：保证用户只能访问自己数据
- 前置条件：请求带有认证用户
- 后置条件：只返回 auth.uid 对应数据
- 基本交互序列：

- 1.接口发起查询。
- 2.数据库执行 RLS 策略。
- 3.返回当前用户数据。
  - 扩展交互序列：
    - 1.未登录请求被拒绝。
    - 业务说明：RLS 是项目隐私安全的基础。



# 序知 AI 个人体系化知识库 软件需求分析文档

## 目 录

|                    |    |
|--------------------|----|
| 1.运行环境 .....       | 1  |
| 1.1 设备 .....       | 1  |
| 1.2 接口 .....       | 1  |
| 2.软件需求 .....       | 2  |
| 2.1 功能需求 .....     | 2  |
| 2.2 非功能需求 .....    | 3  |
| 2.3 软件总体流程 .....   | 4  |
| 2.4 软件体系结构 .....   | 5  |
| 3.需求分析——功能建模 ..... | 6  |
| 3.1 素材管理模块 .....   | 6  |
| 3.2 AI 解析模块 .....  | 8  |
| 3.3 体系化模块 .....    | 10 |
| 3.4 知识问答模块 .....   | 12 |
| 3.5 数据导出模块 .....   | 14 |

## 1. 运行环境

### 1.1 设备

客户端运行在现代浏览器中，主要面向 PC 端使用。开发环境位于 Windows 本地目录 D:\OrdKnow；服务器端可部署到支持 Node.js 的云平台，数据库和认证服务由 Supabase 提供。

### 1.2 接口

- Supabase Auth: 用于用户登录、会话读取和权限判断。
- Supabase PostgreSQL: 用于保存素材、解析结果、知识节点、关系和版本。
- pgvector: 用于保存 embedding 并支持相似内容检索。
- AI Provider: 用于素材解析、体系化重构和知识问答。
- Embedding Provider: 用于长文本分块后的语义向量生成。
- 文件解析/OCR/转写接口: 用于把多模态素材转为文本。

## 2. 软件需求

### 2.1 功能需求

| 模块    | 输入            | 处理                 | 输出                          |
|-------|---------------|--------------------|-----------------------------|
| 素材管理  | 用户文本、文件、图片、音频 | 保存原始内容，维护状态        | materials 记录                |
| AI 解析 | 单条素材          | 调用模型生成结构化 JSON     | material_analysis 记录        |
| 文本分块  | 长文本素材         | 按长度切分并生成 embedding | material_chunks 记录          |
| 体系化   | 已解析素材集合       | 聚类、排序、建层级、建引用      | topics、nodes、links、versions |
| 知识网络  | 节点和来源引用       | 生成节点间关系            | knowledge_edges             |
| 问答    | 用户问题          | 检索知识上下文并调用 AI      | 回答文本                        |
| 导出    | 用户数据          | 组装 JSON 或 Markdown | 下载文件                        |

### 2.2 非功能需求

- 性能需求: 普通页面加载应满足日常交互；AI 类任务允许较长等待，但必须有状态反馈。
- 安全需求: 接口必须验证登录用户，数据库通过 RLS 限制跨用户访问。
- 完整性需求: AI 生成节点必须保留来源素材 ID，避免知识体系失去事实依据。
- 可恢复需求: 体系化失败时 reconstruction\_jobs 保存错误信息，原始素材不受影响。

- 可扩展需求：多模态输入最终统一进入素材层，后续扩展不会破坏核心流程。

2.3 软件总体流程

序知软件总体流程图

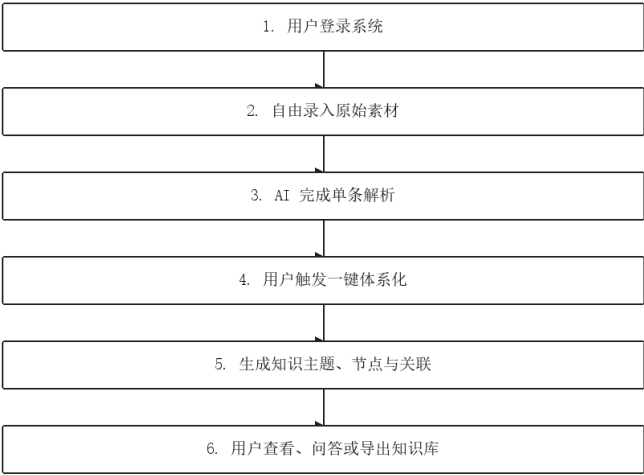


图 2 软件总体流程图

2.4 软件体系结构

序知软件体系结构图

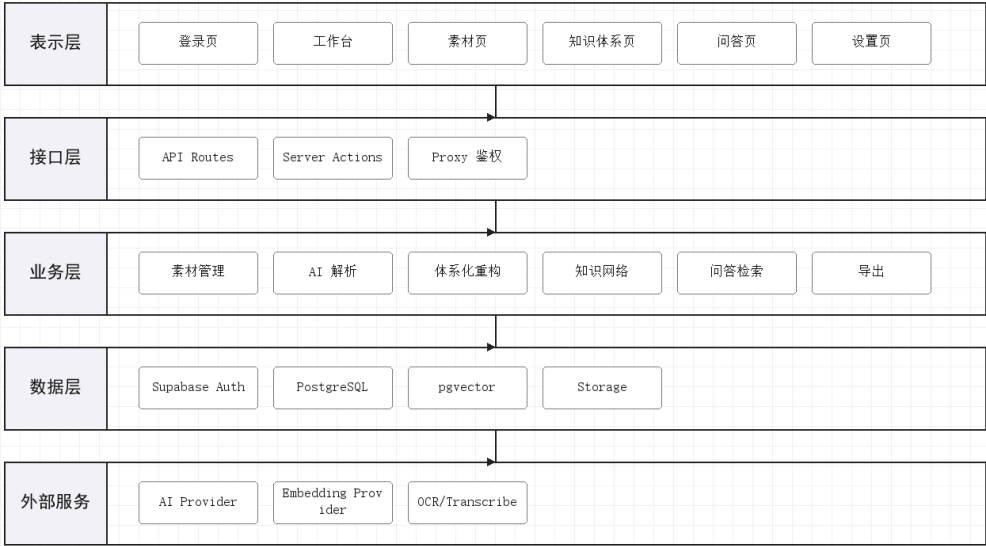


图 3 软件体系结构图

3.需求分析——功能建模

### 3.1 素材管理模块

素材管理模块的核心是“无约束输入”。用户不需要预先判断素材类型或所属主题，系统只负责保存原始内容和状态。

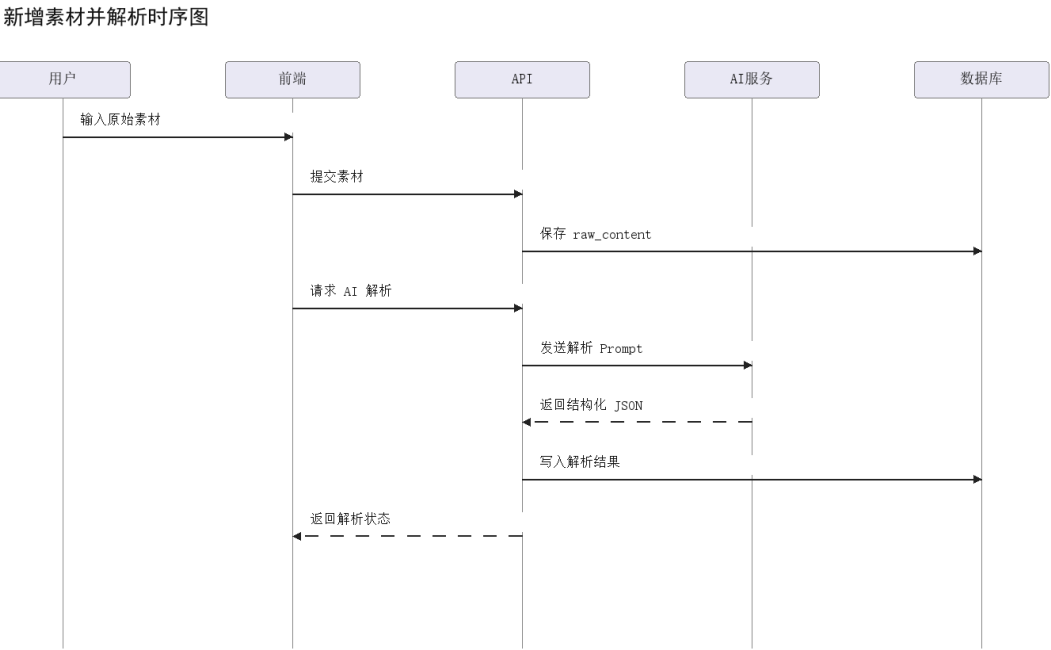


图 4 新增素材并解析时序图

### 3.2 AI 解析模块

AI 解析模块负责将单条原始素材转为结构化理解结果。解析结果不直接替代原文，而是作为后续体系化重构的中间层。

文件导入并解析时序图

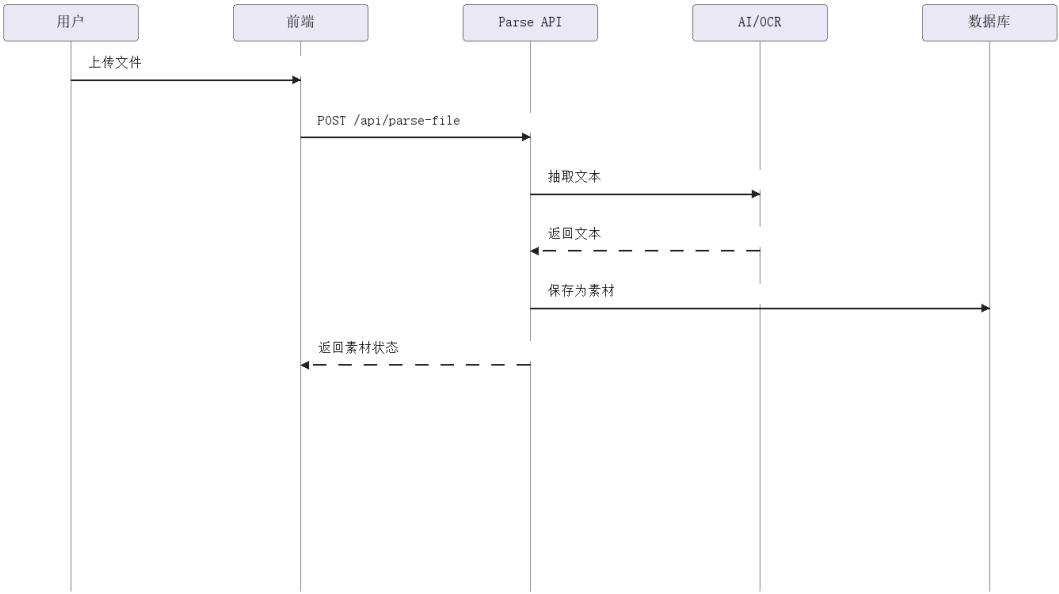


图 5 文件导入并解析时序图

3.3 体系化模块

体系化模块读取当前用户已解析素材，调用 AI 生成知识体系 JSON，并按主题、分支、节点、引用、关系和版本的顺序落库。

一键体系化重构时序图

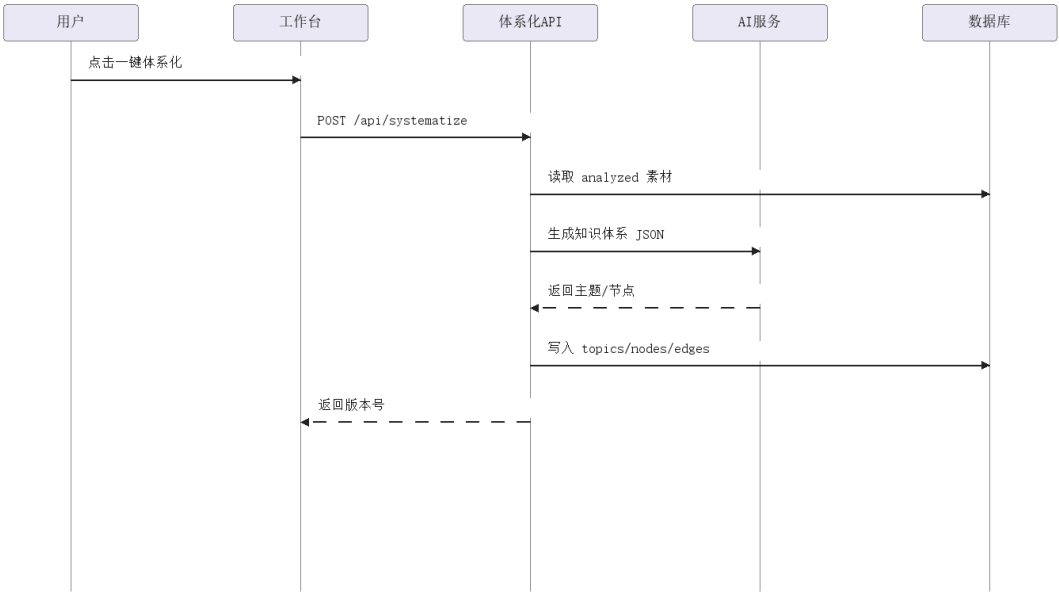


图 6 一键体系化重构时序图

3.4 知识问答模块

知识问答模块不是开放域聊天，而是从用户知识库中组织上下文，再调用 AI 回答。服务端不信任前端传入的素材上下文，而是根据当前用户重新读取数据库。

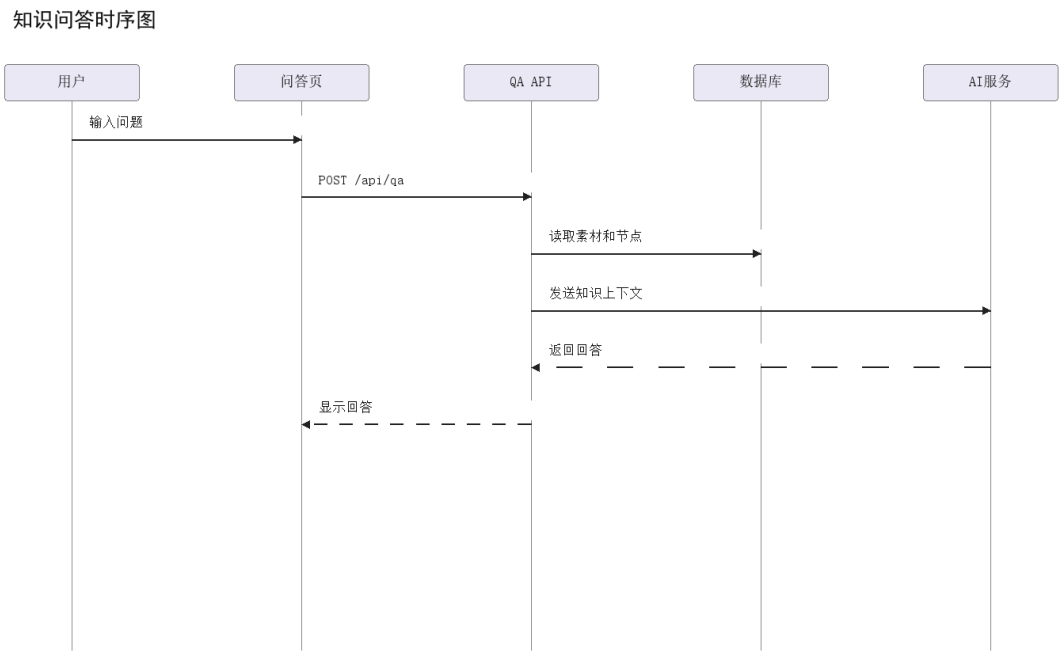


图 7 知识问答时序图

3.5 数据导出模块

导出模块支持 JSON 和 Markdown 两类结果。JSON 适合备份和迁移，Markdown 适合用户阅读、归档或导入其他知识工具。

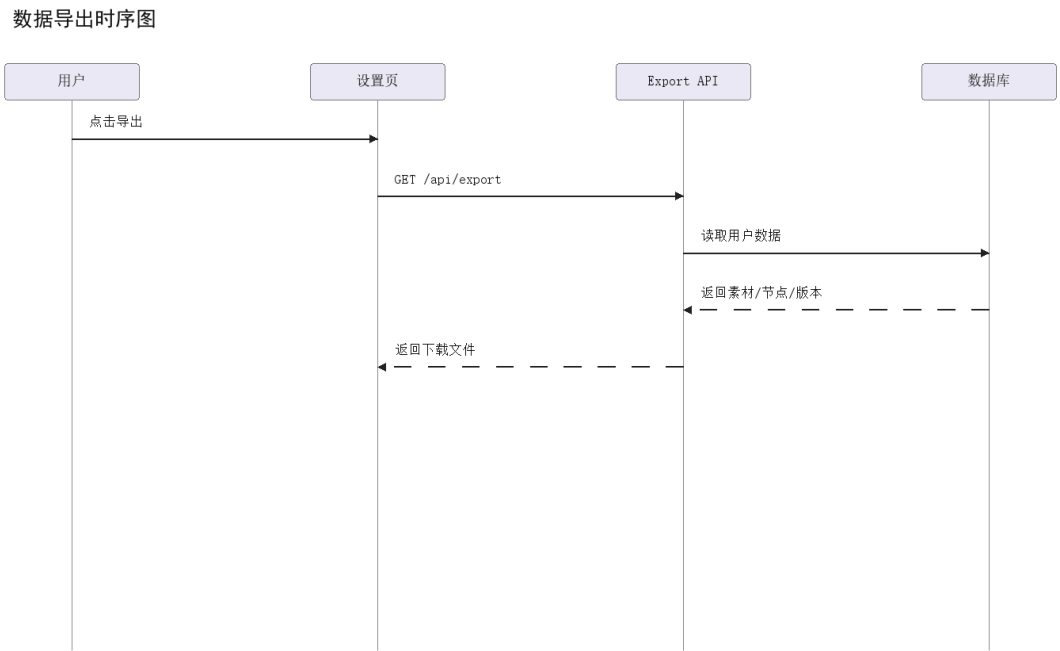


图 8 数据导出时序图

3.6 状态与异常分析

| 对象   | 状态        | 说明      | 异常处理              |
|------|-----------|---------|-------------------|
| 素材   | pending   | 等待解析    | 用户可手动触发解析。        |
| 素材   | analyzing | 正在调用 AI | 超时或失败后转 failed。   |
| 素材   | analyzed  | 解析完成    | 可参与体系化。           |
| 素材   | failed    | 解析失败    | 允许重新解析。           |
| 重构任务 | running   | 正在体系化   | 记录开始时间。           |
| 重构任务 | completed | 体系化成功   | 保存版本快照。           |
| 重构任务 | failed    | 体系化失败   | 记录 error_message。 |



# 序知 AI 个人体系化知识库 软件设计与实现文档

# 目 录

|                 |    |
|-----------------|----|
| 1.系统总体设计 .....  | 1  |
| 1.1 设计原则 .....  | 1  |
| 1.2 总体架构 .....  | 2  |
| 2.数据库设计 .....   | 3  |
| 2.1 数据表关系 ..... | 3  |
| 2.2 数据字典 .....  | 4  |
| 3.接口设计 .....    | 9  |
| 3.1 用户接口 .....  | 9  |
| 3.2 AI 接口 ..... | 11 |
| 3.3 知识接口 .....  | 12 |
| 4.模块设计 .....    | 14 |
| 5.详细设计与实现 ..... | 18 |
| 6.代码结构说明 .....  | 30 |
| 7.部署设计 .....    | 36 |

## 1.系统总体设计

### 1.1 设计原则

- 分层原则：将原始素材层、AI 理解层和知识网络层分开设计。
- 可追溯原则：任何知识节点都应能追溯来源素材。
- 安全原则：用户身份由 Supabase Auth 管理，业务表使用 RLS。
- 可扩展原则：多模态输入最终统一转为文本素材，便于后续扩展。
- 简单可控原则：MVP 阶段体系化采用全量重建，降低实现复杂度。

### 1.2 总体架构

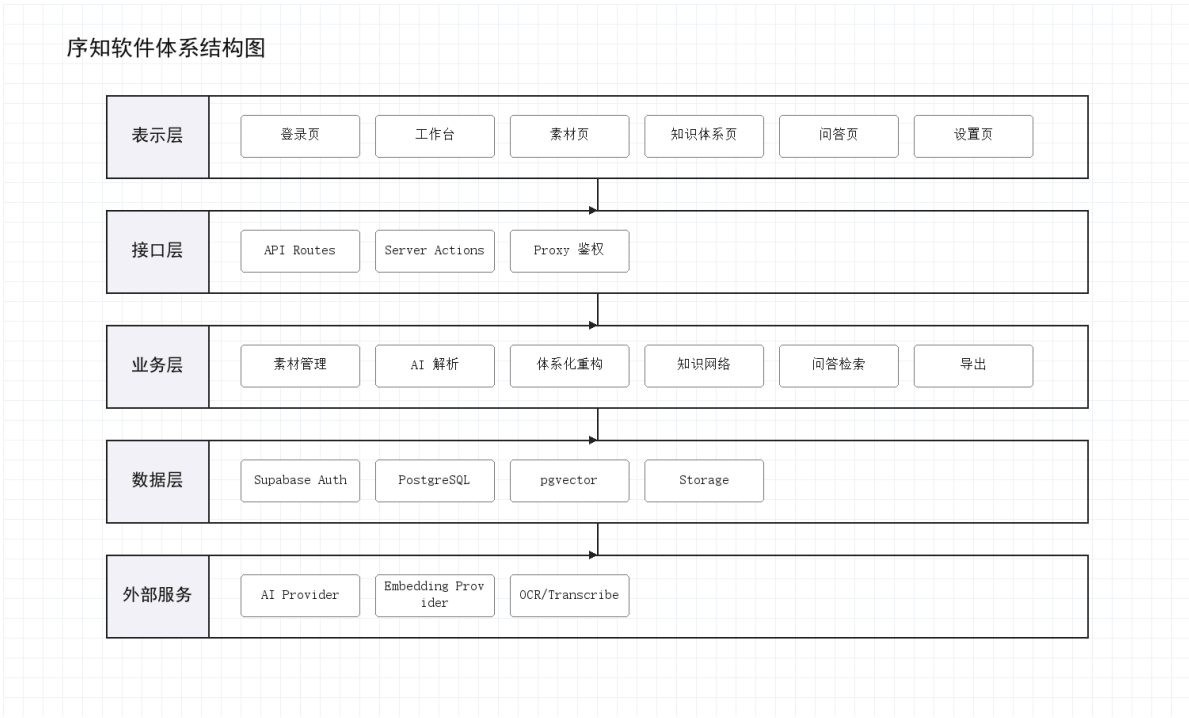


图 9 系统总体架构图

## 2.数据库设计

### 2.1 数据表关系

序知核心数据表关系图

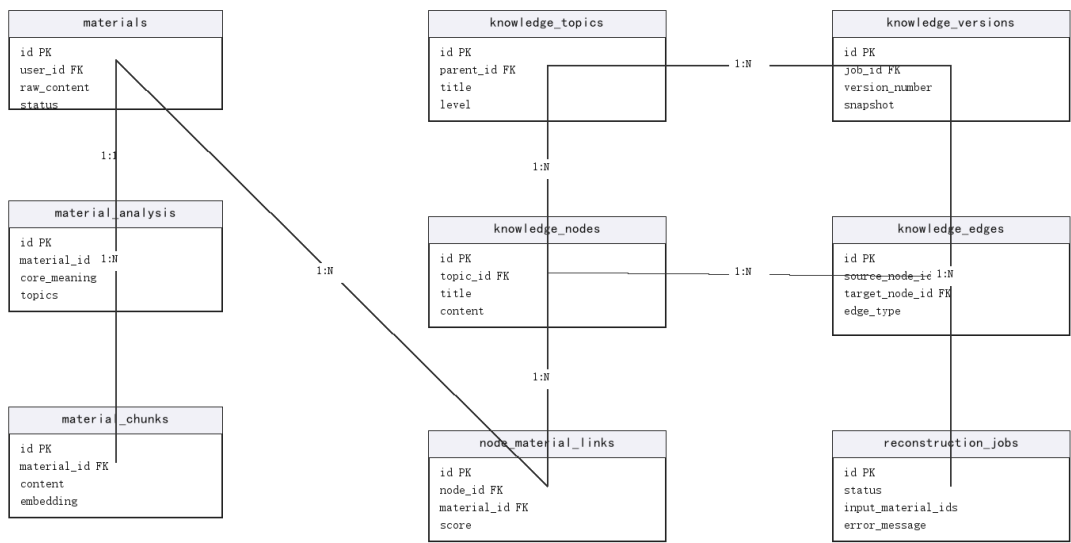


图 10 核心数据表关系图

2.2 数据字典

表 1 materials 数据字典

| 字段          | 类型         | 说明   |
|-------------|------------|------|
| id          | uuid       | 主键   |
| user_id     | uuid       | 所属用户 |
| title       | text       | 素材标题 |
| raw_content | text       | 原始内容 |
| source_type | text       | 素材来源 |
| status      | text       | 处理状态 |
| created_at  | timestampz | 创建时间 |
| updated_at  | timestampz | 更新时间 |

表 2 material\_analysis 数据字典

| 字段               | 类型    | 说明   |
|------------------|-------|------|
| <b>id</b>        | uuid  | 主键   |
| material_id      | uuid  | 关联素材 |
| core_meaning     | text  | 核心含义 |
| useful_points    | jsonb | 有效信息 |
| redundant_points | jsonb | 冗余信息 |
| <b>topics</b>    | jsonb | 主题   |
| knowledge_type   | text  | 知识类型 |
| keywords         | jsonb | 关键词  |
| ai_model         | text  | 使用模型 |

表 3 material\_chunks 数据字典

| 字段             | 类型           | 说明   |
|----------------|--------------|------|
| <b>id</b>      | uuid         | 主键   |
| material_id    | uuid         | 关联素材 |
| chunk_index    | integer      | 分块序号 |
| <b>content</b> | text         | 分块内容 |
| embedding      | vector(1024) | 向量表示 |

表 4 knowledge\_topics 数据字典

| 字段             | 类型   | 说明   |
|----------------|------|------|
| <b>id</b>      | uuid | 主键   |
| <b>user_id</b> | uuid | 所属用户 |

|             |         |      |
|-------------|---------|------|
| parent_id   | uuid    | 父主题  |
| title       | text    | 主题标题 |
| description | text    | 主题说明 |
| level       | integer | 层级   |
| sort_order  | integer | 排序   |

表 5 knowledge\_nodes 数据字典

| 字段         | 类型   | 说明   |
|------------|------|------|
| id         | uuid | 主键   |
| topic_id   | uuid | 所属主题 |
| title      | text | 节点标题 |
| content    | text | 节点内容 |
| summary    | text | 摘要   |
| node_type  | text | 节点类型 |
| version_id | uuid | 版本号  |

表 6 node\_material\_links 数据字典

| 字段              | 类型      | 说明   |
|-----------------|---------|------|
| id              | uuid    | 主键   |
| node_id         | uuid    | 知识节点 |
| material_id     | uuid    | 来源素材 |
| chunk_id        | uuid    | 来源分块 |
| relevance_score | numeric | 相关度  |

表 7 knowledge\_edges 数据字典

| 字段             | 类型      | 说明   |
|----------------|---------|------|
| id             | uuid    | 主键   |
| source_node_id | uuid    | 源节点  |
| target_node_id | uuid    | 目标节点 |
| edge_type      | text    | 关系类型 |
| description    | text    | 关系说明 |
| confidence     | numeric | 置信度  |

表 8 reconstruction\_jobs 与 versions 数据字典

| 字段                          | 类型      | 说明     |
|-----------------------------|---------|--------|
| reconstruction_jobs.status  | text    | 任务状态   |
| input_material_ids          | jsonb   | 输入素材集合 |
| error_message               | text    | 失败原因   |
| knowledge_versions.snapshot | jsonb   | 体系快照   |
| version_number              | integer | 版本号    |

3.接口设计

表 9 主要接口设计表

| 接口                  | 方法   | 功能     |
|---------------------|------|--------|
| /api/materials      | GET  | 查询素材列表 |
| /api/materials      | POST | 新增素材   |
| /api/materials/[id] | GET  | 查看素材详情 |

|                                    |           |                |
|------------------------------------|-----------|----------------|
| /api/materials/[id]                | PATCH     | 编辑素材           |
| /api/materials/[id]                | DELETE    | 删除素材           |
| /api/analyze                       | POST      | AI 单条解析        |
| /api/parse-file                    | POST      | 解析 PDF/Word 文件 |
| /api/ocr                           | POST      | 图片 OCR         |
| /api/transcribe                    | POST      | 音频转写           |
| /api/audio2text                    | POST      | 音频转文本          |
| /api/fetch-url                     | POST      | 抓取网页内容         |
| /api/systematize                   | POST      | 一键体系化          |
| /api/systematize                   | GET       | 查看重构任务         |
| /api/knowledge                     | GET       | 获取知识主题树        |
| /api/knowledge/nodes/all           | GET       | 获取全部节点         |
| /api/knowledge/nodes/[id]          | GET/PATCH | 节点详情与更新        |
| /api/knowledge/node/[id]/materials | GET       | 节点来源素材         |
| /api/knowledge/edges/all           | GET       | 全部知识关系         |
| /api/knowledge/edges/[nodeId]      | GET       | 节点相关关系         |
| /api/knowledge/health              | GET       | 知识库健康检查        |
| /api/knowledge/versions            | GET       | 版本列表           |
| /api/knowledge/versions/[id]/diff  | GET       | 版本差异           |
| /api/qa                            | POST      | 知识问答           |
| /api/search                        | GET       | 搜索素材和知识        |



|                      |      |             |
|----------------------|------|-------------|
| /api/export          | GET  | JSON 导出     |
| /api/export/markdown | GET  | Markdown 导出 |
| /api/setup-storage   | POST | 初始化存储桶      |

4.模块设计

表 10 模块设计表

| 模块    | 主要组成                                                 | 说明                   |
|-------|------------------------------------------------------|----------------------|
| 认证模块  | 登录页、Supabase 客户端、Proxy                               | 完成登录、会话读取和页面保护。      |
| 素材模块  | material-input、material-list、material-detail         | 完成素材 CRUD 和状态展示。     |
| AI 模块 | ai/client、ai/request、ai/analyze、ai/systematize       | 封装模型选择、Key 覆盖、解析和重构。 |
| 知识模块  | knowledge-tree、knowledge-node-detail、knowledge-graph | 展示知识树、节点详情和关系。       |
| 问答模块  | qa-chat、/api/qa                                      | 组织知识上下文并生成回答。        |
| 导出模块  | /api/export、/api/export/markdown                     | 导出用户完整数据和知识体系。       |

4.1 接口逐项说明

4.1.1 GET /api/materials

- 接口功能：查询素材列表
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

4.1.2 POST /api/materials

- 接口功能：新增素材
- 调用位置：由前端页面、服务端组件或设置页操作触发。

- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.3 GET /api/materials/[id]

- 接口功能：查看素材详情
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.4 PATCH /api/materials/[id]

- 接口功能：编辑素材
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.5 DELETE /api/materials/[id]

- 接口功能：删除素材
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.6 POST /api/analyze

- 接口功能：AI 单条解析
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.7 POST /api/parse-file

- 接口功能：解析 PDF/Word 文件
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.8 POST /api/ocr

- 接口功能：图片 OCR
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.9 POST /api/transcribe

- 接口功能：音频转写
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.10 POST /api/audio2text

- 接口功能：音频转文本
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.11 POST /api/fetch-url

- 接口功能：抓取网页内容
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。

- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.12 POST /api/systematize

- 接口功能：一键体系化
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.13 GET /api/systematize

- 接口功能：查看重构任务
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.14 GET /api/knowledge

- 接口功能：获取知识主题树
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.15 GET /api/knowledge/nodes/all

- 接口功能：获取全部节点
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.16 GET/PATCH /api/knowledge/nodes/[id]

- 接口功能：节点详情与更新
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.17 GET /api/knowledge/node/[id]/materials

- 接口功能：节点来源素材
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.18 GET /api/knowledge/edges/all

- 接口功能：全部知识关系
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.19 GET /api/knowledge/edges/[nodeId]

- 接口功能：节点相关关系
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.20 GET /api/knowledge/health

- 接口功能：知识库健康检查
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。

- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.21 GET /api/knowledge/versions

- 接口功能：版本列表
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.22 GET /api/knowledge/versions/[id]/diff

- 接口功能：版本差异
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.23 POST /api/qa

- 接口功能：知识问答
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.24 GET /api/search

- 接口功能：搜索素材和知识
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.25 GET /api/export

- 接口功能：JSON 导出

- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.26 GET /api/export/markdown

- 接口功能：Markdown 导出
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

#### 4.1.27 POST /api/setup-storage

- 接口功能：初始化存储桶
- 调用位置：由前端页面、服务端组件或设置页操作触发。
- 权限要求：除公开登录相关流程外，业务接口均要求用户已登录，并在服务端读取 Supabase 当前用户。
- 输入数据：根据接口类型接收 JSON 请求体、URL 参数、文件表单或查询参数。
- 输出数据：统一返回 JSON 响应，导出接口返回可下载文本或结构化数据。
- 异常处理：未登录返回 401，数据库错误返回 500，业务条件不满足返回 400 或空状态提示。

## 6. 代码结构说明

### 6.1 src/app

Next.js App Router 页面、布局和 API Routes 所在目录。页面层负责路由结构，API 层负责服务端业务逻辑。

这一层靠近路由和页面，最容易把界面逻辑、接口调用和权限判断混在一起。项目中把页面展示和服务端 API 分开写，主要是为了后续排查问题时能很快判断：问题发生在页面、接口，还是数据库。

- 维护重点：保持职责单一，不把 AI 处理、数据库读写和界面展示塞进同一个文件。
- 和产品目标的关系：所有目录最终都围绕“原始素材入库—AI 理解—体系化重构—知识复用”这条主线工作。

### 6.2 src/app/(auth)/login

登录页、注册表单和邮箱密码登录动作。该模块是用户进入系统的身份入口。

这一层靠近路由和页面，最容易把界面逻辑、接口调用和权限判断混在一起。项目中把页面展示和服务端 API 分开写，主要是为了后续排查问题时能很快判断：问题发生在页面、接口，还是数据库。

- 维护重点：保持职责单一，不把 AI 处理、数据库读写和界面展示塞进同一个文件。
- 和产品目标的关系：所有目录最终都围绕“原始素材入库—AI 理解—体系化重构—知识复用”这条主线工作。

### 6.3 src/app/(main)/workspace

工作台页面。三栏结构承载素材输入、素材详情和知识树浏览。

这一层靠近路由和页面，最容易把界面逻辑、接口调用和权限判断混在一起。项目中把页面展示和服务端 API 分开写，主要是为了后续排查问题时能很快判断：问题发生在页面、接口，还是数据库。

- 维护重点：保持职责单一，不把 AI 处理、数据库读写和界面展示塞进同一个文件。
- 和产品目标的关系：所有目录最终都围绕“原始素材入库—AI 理解—体系化重构—知识复用”这条主线工作。

### 6.4 src/app/(main)/materials

素材管理页面。用于素材列表、搜索、状态筛选和详情查看。

这一层靠近路由和页面，最容易把界面逻辑、接口调用和权限判断混在一起。项目中把页面展示和服务端 API 分开写，主要是为了后续排查问题时能很快判断：问题发生在页面、接口，还是数据库。

- 维护重点：保持职责单一，不把 AI 处理、数据库读写和界面展示塞进同一个文件。
- 和产品目标的关系：所有目录最终都围绕“原始素材入库—AI 理解—体系化重构—知识复用”这条主线工作。

### 6.5 src/app/(main)/knowledge

知识体系页面。用于展示 AI 体系化结果、节点详情和知识关系。

这一层靠近路由和页面，最容易把界面逻辑、接口调用和权限判断混在一起。项目中把页面展示和服务端 API 分开写，主要是为了后续排查问题时能很快判断：问题发生在页面、接口，还是数据库。

- 维护重点：保持职责单一，不把 AI 处理、数据库读写和界面展示塞进同一个文件。
- 和产品目标的关系：所有目录最终都围绕“原始素材入库—AI 理解—体系化重构—知识复用”这条主线工作。

### 6.6 src/app/(main)/qa

知识问答页面。用于向个人知识库提问并展示 AI 回答。

这一层靠近路由和页面，最容易把界面逻辑、接口调用和权限判断混在一起。项目中把页面展示和服务端 API 分开写，主要是为了后续排查问题时能很快判断：问题发生在页面、接口，还是数据库。

- 维护重点：保持职责单一，不把 AI 处理、数据库读写和界面展示塞进同一个文件。



- 和产品目标的关系：所有目录最终都围绕“原始素材入库—AI 理解—体系化重构—知识复用”这条主线工作。

## 6.7 src/app/(main)/settings

设置页面。用于模型选择、API Key、导出和隐私说明。

这一层靠近路由和页面，最容易把界面逻辑、接口调用和权限判断混在一起。项目中把页面展示和服务端 API 分开写，主要是为了后续排查问题时能很快判断：问题发生在页面、接口，还是数据库。

- 维护重点：保持职责单一，不把 AI 处理、数据库读写和界面展示塞进同一个文件。
- 和产品目标的关系：所有目录最终都围绕“原始素材入库—AI 理解—体系化重构—知识复用”这条主线工作。

## 6.8 src/components/materials

素材相关组件，包括输入框、列表、详情和状态徽章。

组件目录更偏向交互细节。比如素材输入、知识树、问答框这些内容，都应该让用户一眼知道自己正在处理哪一类知识数据，而不是把业务状态藏在接口返回里。

- 维护重点：保持职责单一，不把 AI 处理、数据库读写和界面展示塞进同一个文件。
- 和产品目标的关系：所有目录最终都围绕“原始素材入库—AI 理解—体系化重构—知识复用”这条主线工作。

## 6.9 src/components/knowledge

知识树、节点详情和图谱组件。

组件目录更偏向交互细节。比如素材输入、知识树、问答框这些内容，都应该让用户一眼知道自己正在处理哪一类知识数据，而不是把业务状态藏在接口返回里。

- 维护重点：保持职责单一，不把 AI 处理、数据库读写和界面展示塞进同一个文件。
- 和产品目标的关系：所有目录最终都围绕“原始素材入库—AI 理解—体系化重构—知识复用”这条主线工作。

## 6.10 src/components/qa

问答聊天组件。

组件目录更偏向交互细节。比如素材输入、知识树、问答框这些内容，都应该让用户一眼知道自己正在处理哪一类知识数据，而不是把业务状态藏在接口返回里。

- 维护重点：保持职责单一，不把 AI 处理、数据库读写和界面展示塞进同一个文件。
- 和产品目标的关系：所有目录最终都围绕“原始素材入库—AI 理解—体系化重构—知识复用”这条主线工作。

## 6.11 src/components/settings

设置页主体组件。

组件目录更偏向交互细节。比如素材输入、知识树、问答框这些内容，都应该让用户一眼知道自己正在处理哪一类知识数据，而不是把业务状态藏在接口返回里。

- 维护重点：保持职责单一，不把 AI 处理、数据库读写和界面展示塞进同一个文件。

- 和产品目标的关系：所有目录最终都围绕“原始素材入库—AI 理解—体系化重构—知识复用”这条主线工作。

## 6.12 src/lib/ai

AI 客户端、Prompt、请求封装、素材解析和体系化重构逻辑。

lib 目录放的是可复用能力。AI 请求、Supabase 客户端、Embedding 分块都在这里集中处理，可以避免同一套鉴权、错误处理和模型选择逻辑散落在多个页面中。

- 维护重点：保持职责单一，不把 AI 处理、数据库读写和界面展示塞进同一个文件。
- 和产品目标的关系：所有目录最终都围绕“原始素材入库—AI 理解—体系化重构—知识复用”这条主线工作。

## 6.13 src/lib/embeddings

文本分块和向量生成逻辑。

lib 目录放的是可复用能力。AI 请求、Supabase 客户端、Embedding 分块都在这里集中处理，可以避免同一套鉴权、错误处理和模型选择逻辑散落在多个页面中。

- 维护重点：保持职责单一，不把 AI 处理、数据库读写和界面展示塞进同一个文件。
- 和产品目标的关系：所有目录最终都围绕“原始素材入库—AI 理解—体系化重构—知识复用”这条主线工作。

## 6.14 src/lib/supabase

Supabase 浏览器端、服务端和中间件封装。

lib 目录放的是可复用能力。AI 请求、Supabase 客户端、Embedding 分块都在这里集中处理，可以避免同一套鉴权、错误处理和模型选择逻辑散落在多个页面中。

- 维护重点：保持职责单一，不把 AI 处理、数据库读写和界面展示塞进同一个文件。
- 和产品目标的关系：所有目录最终都围绕“原始素材入库—AI 理解—体系化重构—知识复用”这条主线工作。

## 6.15 supabase/migrations

数据库扩展、数据表、RLS 策略、索引和触发器迁移文件。

该目录更偏工程支撑，主要服务数据库迁移、文档生成和项目交付。它不直接面对用户，但会影响项目是否能被重新部署、复现和验收。

- 维护重点：保持职责单一，不把 AI 处理、数据库读写和界面展示塞进同一个文件。
- 和产品目标的关系：所有目录最终都围绕“原始素材入库—AI 理解—体系化重构—知识复用”这条主线工作。

## 6.16 tools

报告生成脚本和工程文档自动化工具。

该目录更偏工程支撑，主要服务数据库迁移、文档生成和项目交付。它不直接面对用户，但会影响项目是否能被重新部署、复现和验收。

- 维护重点：保持职责单一，不把 AI 处理、数据库读写和界面展示塞进同一个文件。

- 和产品目标的关系：所有目录最终都围绕“原始素材入库—AI 理解—体系化重构—知识复用”这条主线工作。

7.部署设计

项目代码放在 GitHub 仓库 <https://github.com/usedare/ordknow>，线上访问地址为 <https://ordknow.vercel.app>。部署链路采用 GitHub 推送触发 Vercel 构建的方式，实际操作比较直接：本地通过类型检查和生产构建后提交代码，Vercel 读取仓库并完成线上构建。

这套部署方式的好处是交付痕迹比较清楚：代码提交、构建输出、线上页面访问都能留下证据。它的风险也很明确，主要集中在环境变量和外部服务连通性上。比如 Supabase URL、anon key、AI Provider Key 任意一项缺失，页面虽然能打开，但登录或 AI 功能会失败。

表 10-17 部署配置摘要

| 部署项         | 说明                                                                                  |
|-------------|-------------------------------------------------------------------------------------|
| GitHub 仓库   | <a href="https://github.com/usedare/ordknow">https://github.com/usedare/ordknow</a> |
| Vercel 线上地址 | <a href="https://ordknow.vercel.app">https://ordknow.vercel.app</a>                 |
| 构建命令        | npm run build                                                                       |
| 类型检查        | npm run lint                                                                        |
| 环境变量        | Supabase URL、Supabase anon key、AI Provider key、Embedding key 等。                     |
| 数据库迁移       | 按 supabase/migrations 中 00001 至 00007 顺序执行。                                         |

5.详细设计与实现

1.素材入库详细设计

素材入库模块负责接收用户的自由输入。用户输入的内容可能没有标题、没有分段、存在重复或语序混乱，系统不要求用户整理，只负责保存事实来源。

新增素材时，前端将 title、raw\_content、source\_type 发送给 /api/materials。服务端验证用户身份后写入 materials 表，默认状态为 pending。

编辑素材时，系统只修改当前用户拥有的素材。删除素材时，数据库通过外键级联删除解析结果和分块数据，避免产生孤立记录。

- 入口尽量轻：用户只需要把内容放进来，不承担分类和排版工作。
- 原文不被覆盖：raw\_content 是后续解析、追溯和纠错的依据。

2.AI 单条解析详细设计

AI 单条解析的目标不是写文章，而是提取素材中已经存在的知识信息。系统 Prompt 明确要求不编造外部内容，不做无意义扩写。

解析输出采用固定 JSON 字段：core\_meaning、useful\_points、redundant\_points、topics、knowledge\_type、keywords、related\_hints。这样前端展示和数据库落库都可以保持稳定。

解析成功后，系统将结果写入 material\_analysis，并把素材状态改为 analyzed；解析失败则写入 failed，用户可以重新触发。

- 解析结果是中间层，不直接替代原始素材。
- JSON 字段保持稳定，前端展示和数据库写入才不会因为模型表达变化而失控。

### 3.体系化重构详细设计

体系化接口是系统核心。用户点击一键体系化后，系统创建 reconstruction\_job，并读取当前用户所有 analyzed 素材和解析结果。

MVP 阶段采用全量重建策略：重构前清空当前用户旧的知识节点和关系，然后写入新版主题、分支、节点、来源引用和版本快照。该策略简单、可控，适合课程项目和第一版产品验证。

AI 返回的每个节点必须包含 source\_material\_ids。系统据此写入 node\_material\_links，使用户在知识节点页面可以查看原始素材依据。

- 全量重建适合第一版：实现简单，验收路径清楚，后续可以再升级为增量重构。
- source\_material\_ids 是硬要求，缺少来源的知识节点不能算真正完成。

一键体系化重构时序图

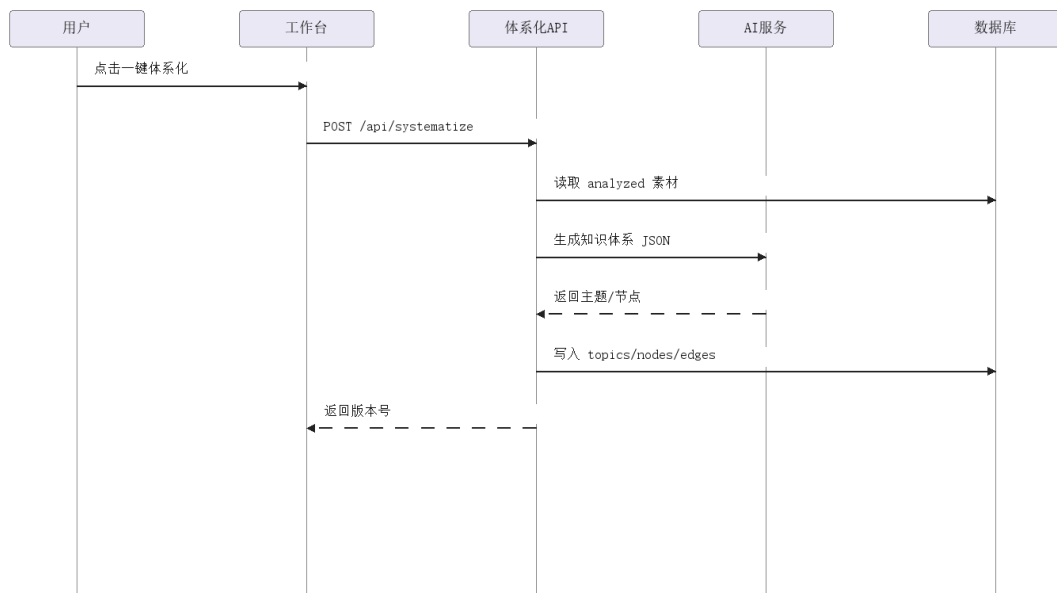


图 11 体系化重构详细时序图

### 4.知识网络详细设计

知识网络由 `knowledge_topics`、`knowledge_nodes`、`node_material_links` 和 `knowledge_edges` 共同组成。`topics` 负责层级，`nodes` 负责内容，`links` 负责来源，`edges` 负责横向关系。

当前实现会根据共同来源素材自动生成 `related` 边。同一条素材支撑多个节点时，这些节点之间天然存在关联。后续可由 AI 进一步判断 `prerequisite`、`supports`、`contradicts` 等关系。

知识网络的价值在于让系统不只是生成树状目录，而是逐步形成可维护、可复用、可演化的个人知识结构。

- `topics` 解决层级，`edges` 解决横向关联，两者合起来才像知识网络。
- 当前关系先从共同来源推断，后续再让 AI 细分前置、支撑、矛盾等关系。

## 5. 知识问答详细设计

问答接口由服务端读取当前用户的素材和知识节点，组织上下文后调用 AI。系统不直接信任前端传入的材料，避免用户伪造上下文读取他人数据。

回答内容应基于用户知识库，不做无关闲聊。若知识库为空或上下文不足，应提示用户先录入素材或进行体系化。

问答结果未来可通过 `source_type='qa'` 回存到 `materials`，使一次高质量问答成为知识网络的新素材。

- 问答接口由服务端重新取上下文，不信任前端传入的素材列表。
- 问答不是闲聊入口，回答必须回到用户已有知识库。

知识问答时序图

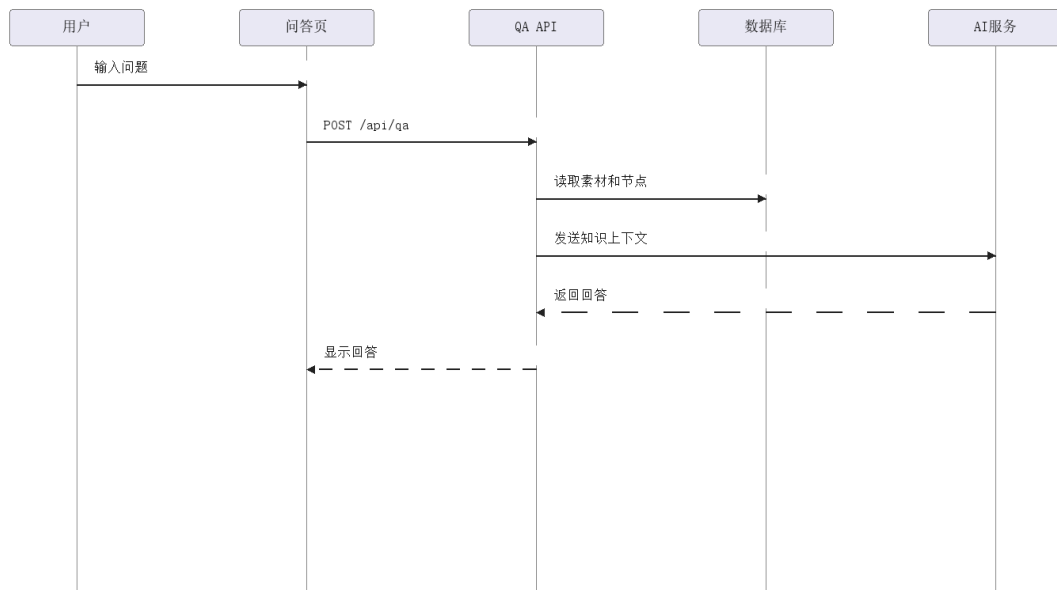


图 12 知识问答详细时序图

## 6. 导出与设置详细设计

设置页负责模型选择、API Key 配置、隐私说明和数据导出。模型选择保存在前端配置中，请求时通过安全头传递到服务端。

JSON 导出包含用户素材、解析结果、知识主题、节点、关系和版本，适合备份和迁移。

Markdown 导出更适合阅读和提交作业附件。

隐私说明强调用户数据存储在 Supabase 项目中，AI 调用会发送必要文本到模型服务，用户可选择自带 API Key。

- 用户 Key 只保存在浏览器 localStorage，不写入数据库。
- 导出功能承担数据可迁移责任，不能只导出页面上看得到的内容。

## 7.异常处理与安全设计

认证异常：未登录用户访问 API 时返回 401，受保护页面通过 Proxy 跳转登录页。

AI 异常：模型调用失败、JSON 格式错误或网络错误时，接口返回错误并更新任务状态。

数据库异常：写入失败时捕获错误并返回 500，体系化任务记录 error\_message。

权限安全：所有业务查询均带 user\_id 过滤，数据库层启用 RLS，形成双重保护。

- 异常状态必须写清楚，失败不能破坏原始素材。
- 权限校验同时放在 API 和 RLS 两层，避免只靠前端控制。

## 8.核心代码实现说明

### 8.1 src/app/api/materials/route.ts

素材列表查询与新增素材接口。该文件负责读取当前登录用户，按 user\_id 查询或写入 materials。

实现时最关键的是先确认用户身份，再处理业务参数。这个顺序不能反过来，否则接口很容易在异常路径里泄露数据。接口返回保持 JSON 为主，下载类接口例外。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.2 src/app/api/materials/[id]/route.ts

素材详情、编辑和删除接口。该接口必须验证素材归属，避免跨用户访问。

实现时最关键的是先确认用户身份，再处理业务参数。这个顺序不能反过来，否则接口很容易在异常路径里泄露数据。接口返回保持 JSON 为主，下载类接口例外。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.3 src/app/api/analyze/route.ts

AI 单条解析接口。负责调用解析逻辑、保存 material\_analysis、生成 chunks 并更新素材状态。

实现时最关键的是先确认用户身份，再处理业务参数。这个顺序不能反过来，否则接口很容易在异常路径里泄露数据。接口返回保持 JSON 为主，下载类接口例外。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.4 src/app/api/systematize/route.ts

一键体系化接口。负责创建 reconstruction\_job，读取 analyzed 素材，调用 AI 体系化并落库。

实现时最关键的是先确认用户身份，再处理业务参数。这个顺序不能反过来，否则接口很容易在异常路径里泄露数据。接口返回保持 JSON 为主，下载类接口例外。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.5 src/app/api/qa/route.ts

知识问答接口。服务端读取当前用户素材和知识节点，组织上下文后调用 AI。

实现时最关键的是先确认用户身份，再处理业务参数。这个顺序不能反过来，否则接口很容易在异常路径里泄露数据。接口返回保持 JSON 为主，下载类接口例外。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.6 src/app/api/knowledge/route.ts

知识树接口。查询 topics、nodes 和 links，返回前端可渲染的树状结构。

实现时最关键的是先确认用户身份，再处理业务参数。这个顺序不能反过来，否则接口很容易在异常路径里泄露数据。接口返回保持 JSON 为主，下载类接口例外。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.7 src/app/api/knowledge/edges/all/route.ts

知识关系接口。返回当前用户的全部知识边，用于图谱展示。

实现时最关键的是先确认用户身份，再处理业务参数。这个顺序不能反过来，否则接口很容易在异常路径里泄露数据。接口返回保持 JSON 为主，下载类接口例外。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.8 src/app/api/knowledge/health/route.ts

知识库健康检查接口。检查重复、孤儿节点、无来源节点等问题。

实现时最关键的是先确认用户身份，再处理业务参数。这个顺序不能反过来，否则接口很容易在异常路径里泄露数据。接口返回保持 JSON 为主，下载类接口例外。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.9 src/app/api/export/route.ts

JSON 导出接口。读取用户完整数据并返回可备份的结构。

实现时最关键的是先确认用户身份，再处理业务参数。这个顺序不能反过来，否则接口很容易在异常路径里泄露数据。接口返回保持 JSON 为主，下载类接口例外。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.10 src/app/api/export/markdown/route.ts

Markdown 导出接口。将知识体系转换为便于阅读和迁移的文本。

实现时最关键的是先确认用户身份，再处理业务参数。这个顺序不能反过来，否则接口很容易在异常路径里泄露数据。接口返回保持 JSON 为主，下载类接口例外。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.11 src/app/api/parse-file/route.ts

文件解析接口。使用 pdf-parse、mammoth 等库把 PDF/Word 转为文本。

实现时最关键的是先确认用户身份，再处理业务参数。这个顺序不能反过来，否则接口很容易在异常路径里泄露数据。接口返回保持 JSON 为主，下载类接口例外。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.12 src/app/api/ocr/route.ts

OCR 接口。将图片中的文字识别为可入库素材。

实现时最关键的是先确认用户身份，再处理业务参数。这个顺序不能反过来，否则接口很容易在异常路径里泄露数据。接口返回保持 JSON 为主，下载类接口例外。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.13 src/app/api/transcribe/route.ts

音频转写接口。将音频内容转化为文本素材。

实现时最关键的是先确认用户身份，再处理业务参数。这个顺序不能反过来，否则接口很容易在异常路径里泄露数据。接口返回保持 JSON 为主，下载类接口例外。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.14 src/lib/ai/client.ts

AI 客户端封装。统一模型调用方式，降低业务接口与具体模型提供方耦合。

这一类文件直接影响 AI 输出质量。代码里把模型调用、Prompt 和业务接口拆开，是为了让后续换模型或调整 Prompt 时不需要重写页面。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.15 src/lib/ai/request.ts

AI 请求配置。读取用户自带 Key 和模型选择，形成服务端调用参数。

这一类文件直接影响 AI 输出质量。代码里把模型调用、Prompt 和业务接口拆开，是为了让后续换模型或调整 Prompt 时不需要重写页面。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。



### 8.16 src/lib/ai/prompts.ts

Prompt 约束。规定 AI 不能脱离用户素材编造，必须输出结构化结果。

这一类文件直接影响 AI 输出质量。代码里把模型调用、Prompt 和业务接口拆开，是为了让后续换模型或调整 Prompt 时不需要重写页面。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.17 src/lib/ai/analyze.ts

单条素材解析逻辑。把原始素材转化为 MaterialAnalysisResult。

这一类文件直接影响 AI 输出质量。代码里把模型调用、Prompt 和业务接口拆开，是为了让后续换模型或调整 Prompt 时不需要重写页面。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.18 src/lib/ai/systematize.ts

体系化重构逻辑。把素材集合转化为 SystematizeResult。

这一类文件直接影响 AI 输出质量。代码里把模型调用、Prompt 和业务接口拆开，是为了让后续换模型或调整 Prompt 时不需要重写页面。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.19 src/lib/embeddings/chunk.ts

文本分块逻辑。控制 chunk 大小，为向量检索和长文本处理服务。

Embedding 相关代码属于检索基础能力。即使当前页面展示还比较轻，分块和向量字段也为后续语义搜索、相似素材推荐、知识关联打下了基础。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.20 src/lib/embeddings/client.ts

Embedding 客户端。负责调用向量模型并处理失败兜底。

Embedding 相关代码属于检索基础能力。即使当前页面展示还比较轻，分块和向量字段也为后续语义搜索、相似素材推荐、知识关联打下了基础。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.21 src/components/workspace/workspace-layout.tsx

工作台主布局。实现左素材、中详情、右知识树的核心使用界面。

组件实现更接近用户实际体验。这里的重点不是炫技，而是让用户能稳定完成输入、查看、提问、导出这些动作。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。

- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.22 src/components/materials/material-input.tsx

素材输入组件。承载用户无约束录入的第一入口。

组件实现更接近用户实际体验。这里的重点不是炫技，而是让用户能稳定完成输入、查看、提问、导出这些动作。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.23 src/components/materials/material-list.tsx

素材列表组件。展示素材状态、标题、时间和筛选结果。

组件实现更接近用户实际体验。这里的重点不是炫技，而是让用户能稳定完成输入、查看、提问、导出这些动作。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.24 src/components/knowledge/knowledge-tree.tsx

知识树组件。展示 AI 体系化后的主题和节点层级。

组件实现更接近用户实际体验。这里的重点不是炫技，而是让用户能稳定完成输入、查看、提问、导出这些动作。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.25 src/components/knowledge/knowledge-node-detail.tsx

节点详情组件。展示内容、来源素材和相关节点。

组件实现更接近用户实际体验。这里的重点不是炫技，而是让用户能稳定完成输入、查看、提问、导出这些动作。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.26 src/components/qa/qa-chat.tsx

问答组件。展示用户问题和 AI 回答。

组件实现更接近用户实际体验。这里的重点不是炫技，而是让用户能稳定完成输入、查看、提问、导出这些动作。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

### 8.27 src/components/settings/settings-content.tsx

设置组件。管理模型、Key、导出和隐私信息。

组件实现更接近用户实际体验。这里的重点不是炫技，而是让用户能稳定完成输入、查看、提问、导出这些动作。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

8.28 src/types/index.ts

核心业务类型。定义 Material、KnowledgeNode、KnowledgeEdge 等数据结构。

该文件承担基础支撑作用。它不一定直接产生页面，但会影响类型边界、权限入口或项目整体可维护性。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

8.29 src/proxy.ts

Next.js 代理/中间件入口。负责受保护路由的会话检查。

该文件承担基础支撑作用。它不一定直接产生页面，但会影响类型边界、权限入口或项目整体可维护性。

- 质量要求：职责清楚、错误处理明确、不破坏原始素材事实源。
- 验收方式：结合源码检查、构建输出、接口访问和页面截图共同判断。

9.AI Prompt 设计

- 你是“序知”的知识解析引擎，不是聊天助手。
- 只能基于用户提供的原始素材提取真实存在的信息。
- 不要添加原文没有的信息，不做外部编造。
- 输出必须是结构化 JSON，字段必须稳定。
- 体系化重构时必须保留 source\_material\_ids。
- 优先生成“总—分—细”的层级结构。
- 发现重复内容时合并，发现矛盾时保留来源而不是擅自消除。

表 9-1 AI Prompt 类型说明

| Prompt 类型   | 输入             | 输出                     | 用途               |
|-------------|----------------|------------------------|------------------|
| 单条解析 Prompt | 单条 raw_content | MaterialAnalysisResult | 形成理解层，减少后续重构噪声。  |
| 体系化 Prompt  | 素材与解析结果集合      | SystematizeResult      | 生成主题、分支、节点和来源引用。 |
| 问答 Prompt   | 用户问题与知识库上下文    | 回答文本                   | 基于个人知识库复用知识。     |
| 健康检查 Prompt | 知识节点和关系        | 问题与建议                  | 发现重复、孤儿、无来源等风险。  |

## 10.数据库安全策略实现

表 10-1 RLS 安全策略表

| 数据表                 | RLS 策略核心                | 安全意义             |
|---------------------|-------------------------|------------------|
| materials           | auth.uid() = user_id    | 用户只能访问自己的原始素材。   |
| material_analysis   | auth.uid() = user_id    | 用户只能访问自己的解析结果。   |
| material_chunks     | auth.uid() = user_id    | 用户只能访问自己的素材分块。   |
| knowledge_topics    | auth.uid() = user_id    | 用户只能访问自己的知识主题。   |
| knowledge_nodes     | auth.uid() = user_id    | 用户只能访问自己的知识节点。   |
| node_material_links | 通过 knowledge_nodes 归属判断 | 引用关系必须属于当前用户的节点。 |
| knowledge_edges     | auth.uid() = user_id    | 知识关系按用户隔离。       |
| reconstruction_jobs | auth.uid() = user_id    | 重构任务按用户隔离。       |
| knowledge_versions  | auth.uid() = user_id    | 版本快照按用户隔离。       |

### 10.1 materials 权限设计

materials 表的权限策略核心是“auth.uid() = user\_id”。该策略的意义是：用户只能访问自己的原始素材。在服务端接口中，系统还会显式按 user\_id 查询，形成应用层和数据库层的双重保护。

### 10.2 material\_analysis 权限设计

material\_analysis 表的权限策略核心是“auth.uid() = user\_id”。该策略的意义是：用户只能访问自己的解析结果。在服务端接口中，系统还会显式按 user\_id 查询，形成应用层和数据库层的双重保护。

### 10.3 material\_chunks 权限设计

material\_chunks 表的权限策略核心是“auth.uid() = user\_id”。该策略的意义是：用户只能访问自己的素材分块。在服务端接口中，系统还会显式按 user\_id 查询，形成应用层和数据库层的双重保护。

### 10.4 knowledge\_topics 权限设计

knowledge\_topics 表的权限策略核心是“auth.uid() = user\_id”。该策略的意义是：用户只能访问自己的知识主题。在服务端接口中，系统还会显式按 user\_id 查询，形成应用层和数据库层的双重保护。

### 10.5 knowledge\_nodes 权限设计

knowledge\_nodes 表的权限策略核心是“auth.uid() = user\_id”。该策略的意义是：用户只能访问自己的知识节点。在服务端接口中，系统还会显式按 user\_id 查询，形成应用层和数据库层的双重保护。

## 10.6 node\_material\_links 权限设计

node\_material\_links 表的权限策略核心是“通过 knowledge\_nodes 归属判断”。该策略的意义是：引用关系必须属于当前用户的节点。在服务端接口中，系统还会显式按 user\_id 查询，形成应用层和数据库层的双重保护。

## 10.7 knowledge\_edges 权限设计

knowledge\_edges 表的权限策略核心是“auth.uid() = user\_id”。该策略的意义是：知识关系按用户隔离。在服务端接口中，系统还会显式按 user\_id 查询，形成应用层和数据库层的双重保护。

## 10.8 reconstruction\_jobs 权限设计

reconstruction\_jobs 表的权限策略核心是“auth.uid() = user\_id”。该策略的意义是：重构任务按用户隔离。在服务端接口中，系统还会显式按 user\_id 查询，形成应用层和数据库层的双重保护。

## 10.9 knowledge\_versions 权限设计

knowledge\_versions 表的权限策略核心是“auth.uid() = user\_id”。该策略的意义是：版本快照按用户隔离。在服务端接口中，系统还会显式按 user\_id 查询，形成应用层和数据库层的双重保护。

# **序知 AI 个人体系化知识库 测试报告与用户使用说明书**

目 录

1.测试目标 ..... 1

2.测试环境 ..... 2

3.功能测试 ..... 3

4.非功能测试 ..... 8

5.运行环境 ..... 10

6.使用说明 ..... 12

6.1 登录界面 ..... 12

6.2 工作台界面 ..... 13

6.3 素材界面 ..... 14

6.4 知识体系界面 ..... 15

6.5 问答界面 ..... 16

6.6 设置界面 ..... 17

1.测试目标

本测试文档用于验证序知是否完成核心软件需求，重点检查素材入库、AI 解析、一键体系化、知识网络、问答和导出功能是否形成闭环。

序知测试流程图

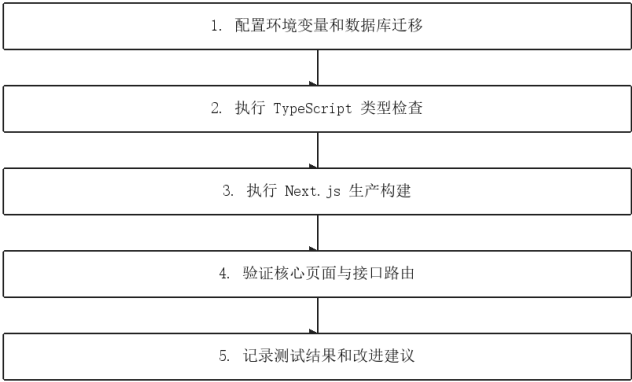


图 13 系统测试流程图

2.测试环境

表 11 测试环境表

| 项目   | 配置                                     |
|------|----------------------------------------|
| 操作系统 | Windows，本地项目目录 D:\OrdKnow              |
| 前端框架 | Next.js 16.2.7， React 19.2.7           |
| 开发语言 | TypeScript 6.0.3                       |
| 数据库  | Supabase PostgreSQL， pgvector          |
| 验证命令 | npm run lint; npm run build            |
| 验证结果 | TypeScript 检查通过；生产构建通过，输出 27 个页面/接口路由。 |



```
npm run lint - TypeScript 类型检查输出

> ordknow@0.1.0 lint
> tsc --noEmit -p tsconfig.typecheck.json
```

图 14 npm run lint 类型检查真实输出

```
npm run build - Next.js 生产构建输出

> ordknow@0.1.0 build
> next build

▲ Next.js 16.2.7 (Turbopack)
- Environments: .env.local

  Creating an optimized production build ...
  Compiled successfully in 8.9s
  Running TypeScript ...
  Finished TypeScript in 8.7s ...
  Collecting page data using 19 workers ...
  Generating static pages using 19 workers (0/27) ...
  Generating static pages using 19 workers (6/27)
  Generating static pages using 19 workers (13/27)
  Generating static pages using 19 workers (20/27)
  Generating static pages using 19 workers (27/27) in 690ms
  Finalizing page optimization ...

Route (app)
├ ○ /
├ f /_not-found
├ f /api/analyze
├ f /api/audio2text
├ f /api/export
├ f /api/export/markdown
├ f /api/fetch-url
├ f /api/knowledge
├ f /api/knowledge/edges/[nodeId]
├ f /api/knowledge/edges/all
├ f /api/knowledge/health
├ f /api/knowledge/node/[id]/materials
├ f /api/knowledge/nodes/[id]
├ f /api/knowledge/nodes/[id]/regenerate
├ f /api/knowledge/nodes/all
├ f /api/knowledge/versions
├ f /api/knowledge/versions/[id]/diff
├ f /api/materials
├ f /api/materials/[id]
├ f /api/ocr
├ f /api/parse-file
├ f /api/qa
├ f /api/search
├ f /api/setup-storage
├ f /api/systematize
├ f /api/transcribe
├ ○ /knowledge
├ f /login
├ ○ /materials
├ ○ /qa
├ ○ /settings
├ ○ /workspace

f Proxy (Middleware)

○ (Static) prerendered as static content
f (Dynamic) server-rendered on demand
```

图 15 npm run build 生产构建真实输出

## 线上访问与未登录保护 HTTP 检查输出

序知线上与权限保护 HTTP 检查

executed\_at=2026-06-15 23:06:40

```
GET https://ordknow.vercel.app -> 200; final=https://ordknow.vercel.app/login; 线上首页可访问; body=<!DOCTYPE html><html lang="zh-CN"><head><meta charset="utf-8"/><meta name="viewport" content="width=device-width, initial-scale=1"></head><body><div></div></body></html>
GET https://ordknow.vercel.app/login -> 200; final=https://ordknow.vercel.app/login; 登录页可访问; body=<!DOCTYPE html><html lang="zh-CN"><head><meta charset="utf-8"/><meta name="viewport" content="width=device-width, initial-scale=1"></head><body><div></div></body></html>
GET https://ordknow.vercel.app/workspace -> 200; final=https://ordknow.vercel.app/login; 未登录工作台会跳转到登录页; body=<!DOCTYPE html><html lang="zh-CN"><head><meta charset="utf-8"/><meta name="viewport" content="width=device-width, initial-scale=1"></head><body><div></div></body></html>
GET https://ordknow.vercel.app/materials -> 200; final=https://ordknow.vercel.app/login; 未登录素材页会跳转到登录页; body=<!DOCTYPE html><html lang="zh-CN"><head><meta charset="utf-8"/><meta name="viewport" content="width=device-width, initial-scale=1"></head><body><div></div></body></html>
GET https://ordknow.vercel.app/api/materials -> 200; final=https://ordknow.vercel.app/login; 未登录素材接口被中间件保护并跳转登录; body=<!DOCTYPE html><html lang="zh-CN"><head><meta charset="utf-8"/><meta name="viewport" content="width=device-width, initial-scale=1"></head><body><div></div></body></html>
GET https://ordknow.vercel.app/api/knowledge -> 200; final=https://ordknow.vercel.app/login; 未登录知识接口被中间件保护并跳转登录; body=<!DOCTYPE html><html lang="zh-CN"><head><meta charset="utf-8"/><meta name="viewport" content="width=device-width, initial-scale=1"></head><body><div></div></body></html>
POST https://ordknow.vercel.app/api/qa -> ERROR URL:Error: <urlopen error [SSL: UNEXPECTED_EOF_WHILE_READING] EOF occurred in violation of protocol (_ssl.c:1010)>; 未登录问答接口返回重定向
POST https://ordknow.vercel.app/api/systematize -> 307; final=https://ordknow.vercel.app/api/systematize; 未登录体系化接口返回重定向; body=Redirecting...
GET https://github.com/usedare/ordknow -> 200; final=https://github.com/usedare/ordknow; GitHub仓库页面可访问; body=<!DOCTYPE html><html lang="en"><head><meta charset="utf-8"/><meta name="viewport" content="width=device-width, initial-scale=1"></head><body><div></div></body></html>
```

图 16 线上访问与未登录保护真实输出

## 测试口径说明

本次测试使用 test@ordknow.com / test123456 作为登录账号。报告只把有命令输出、页面截图或 HTTP 响应支撑的内容写成通过；实际跑不通的地方按真实结果记录。本次登录表单能够正常打开和提交，但线上服务端认证返回 fetch failed，因此没有伪造工作台、素材页、知识页的登录态截图。

## 3.功能测试

测试没有继续堆 60 行同款表格。实际验收按三条线走：第一条是工程线，检查 lint、build、GitHub 和 Vercel；第二条是访问线，检查公开页面和未登录保护；第三条是账号线，用测试账号提交登录表单，观察真实错误。这样写虽然不如“全绿”好看，但更接近本项目当前状态。

表 12 实际执行测试记录

| 编号    | 测试对象    | 实际操作                                                     | 结果                             |
|-------|---------|----------------------------------------------------------|--------------------------------|
| TC-01 | 登录页访问   | 打开 https://ordknow.vercel.app/login                      | 页面正常显示，截图已保存。                  |
| TC-02 | 登录表单填写  | 使用 test@ordknow.com / test123456 填写登录表单                  | 字段填写正确，提交后没有进入工作台。             |
| TC-03 | 登录失败反馈  | 点击登录按钮后等待跳转                                              | 页面显示 fetch failed，说明服务端认证请求失败。 |
| TC-04 | 未登录路由保护 | 直接访问 /workspace、/materials、/api/materials、/api/knowledge | 最终回到 /login 或返回受保护状态。          |

|       |            |                        |                        |
|-------|------------|------------------------|------------------------|
| TC-05 | 类型检查       | 执行 npm run lint        | 通过。                    |
| TC-06 | 生产构建       | 执行 npm run build       | 通过，构建输出包含主要页面和 API 路由。 |
| TC-07 | GitHub 仓库  | 访问 usedare/ordknow     | 仓库可访问，最终代码和文档已推送。      |
| TC-08 | StarUML 图表 | 通过 StarUML MCP 生成并导出图表 | 6 张 UML/ER 图已嵌入报告。     |

3.1 测试执行记录

表 13 自动化实测证据清单

| 证据编号 | 实测对象            | 执行方式                                                                | 实际结果                               |
|------|-----------------|---------------------------------------------------------------------|------------------------------------|
| E-01 | TypeScript 类型检查 | 本地执行 npm run lint                                                   | 通过，输出已保存到 docs/evidence/lint.log。  |
| E-02 | 生产构建            | 本地执行 npm run build                                                  | 通过，Next.js 生成 27 个页面/接口路由。         |
| E-03 | 线上入口            | HTTP GET<br>https://ordknow.vercel.app/login                        | 返回 200，登录页可访问。                     |
| E-04 | 未登录保护           | HTTP 访问 /workspace、<br>/materials、/api/materials、<br>/api/knowledge | 最终跳转到 /login；POST 体系化接口返回 307。     |
| E-05 | GitHub 仓库       | 访问<br>https://github.com/usedare/ordknow                            | 返回 200，代码和报告已推送。                   |
| E-06 | 测试账号登录          | Chrome CDP 自动填写<br>test@ordknow.com / test123456 并提交                | 登录表单可提交，但线上返回 fetch failed，未进入工作台。 |
| E-07 | StarUML 图表      | StarUML MCP 生成并通过本地 API 导出 PNG                                      | 6 张核心图已嵌入报告，保留未注册水印。               |

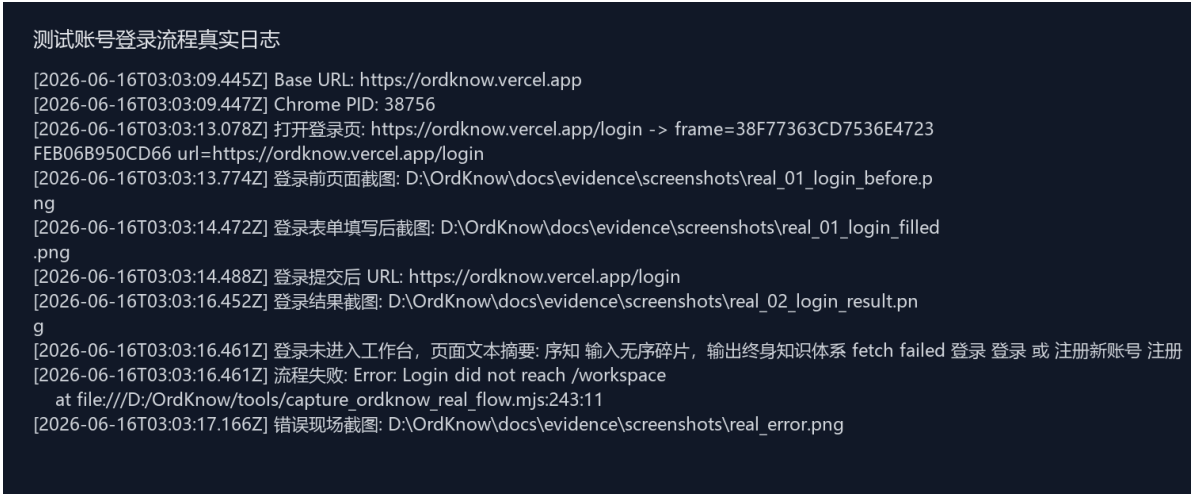


图 17 测试账号登录流程真实日志



图 18 线上登录页初始状态截图

序知

输入无序碎片，输出终身知识体系

登录

test@ordknow.com

\*\*\*\*\*

登录

或

注册新账号

邮箱地址

密码（至少 6 位）

注册

图 19 测试账号填写后截图

序知

输入无序碎片，输出终身知识体系

fetch failed

登录

邮箱地址

密码

登录

或

注册新账号

邮箱地址

密码（至少 6 位）

注册

图 20 登录提交后 fetch failed 结果截图

3.2 已实测项目

62

- 类型检查与生产构建已经由命令行执行，日志和截图均进入 docs/evidence。该部分可证明项目至少在类型层和构建层没有阻塞性错误。
- 线上首页和登录页已通过 HTTP 检查，/login 返回 200。由于根路径会进入登录保护，最终地址显示为 /login，符合当前认证设计。
- 未登录访问受保护页面和部分接口时会被中间件导向登录页，POST 体系化接口返回 307。该结果说明线上部署已经启用认证保护。
- 测试账号已经用于真实登录尝试。表单填写正确，页面能把错误展示出来；失败点不是前端页面打不开，而是服务端认证请求返回 fetch failed。
- GitHub 仓库页面可访问，最终提交已经推送到 main 分支；StarUML 图表由 MCP 生成后导出为真实 PNG。

### 3.3 登录态实测结论

#### 登录链路当前阻塞

使用测试账号提交登录后，线上页面返回 fetch failed。本地直接请求 Supabase Auth 也出现 SSL 连接失败，说明需要优先检查 Vercel 与 Supabase 的网络连通性、Supabase URL/anon key 配置，以及部署环境中 Node fetch 到 Supabase 的 TLS 行为。在这个问题修复前，报告不伪造工作台、素材页、知识页和问答页的登录态截图。

素材入库、AI 解析、一键体系化、知识问答和导出功能的代码路径均已进入构建，但这些页面需要登录态才能继续做端到端验收。

这次补充的真实测试至少确认了两个事实：第一，登录页和表单本身存在；第二，当前阻塞发生在认证服务请求，而不是页面路由或按钮缺失。

后续修复应从 Supabase Auth 连通性开始。登录打通后，再按“新增素材—解析—体系化—问答—导出”的顺序补一轮完整截图。

## 4. 非功能测试

- 安全测试：验证未登录用户不能访问受保护接口，数据库 RLS 不允许跨用户读取。
- 构建测试：执行 npm run build，确认 Next.js 生产构建通过。
- 类型测试：执行 npm run lint，确认 TypeScript 类型检查通过。
- 异常测试：模拟 AI Key 缺失、无 analyzed 素材、文件解析失败等情况。
- 兼容测试：在 Chrome/Edge 浏览器中验证主要页面显示与交互。

## 5. 运行环境

### 5.1 硬件运行环境

开发端建议使用 8GB 以上内存的 PC；生产端可部署在支持 Node.js 的云平台，数据库使用 Supabase 托管。

### 5.2 软件运行环境

- Node.js 与 npm
- Next.js

- Supabase
- PostgreSQL + pgvector
- AI Provider
- Embedding Provider

## 6. 用户使用说明

### 6.1 登录界面

用户第一次进入序知时会先看到登录页。这个页面没有额外导航和宣传内容，只保留产品名称、说明语、登录表单和注册表单。这样的处理比较直接，也符合本产品“先进入个人知识库”的使用方式。

- 常用操作
  1. 输入邮箱。
  2. 输入密码。
  3. 点击登录按钮。
  4. 认证成功后进入工作台。

验收时应同时看页面是否能打开、按钮是否能点击、失败提示是否清楚，以及操作结果有没有真正回到个人知识库数据。

### 6.2 工作台界面

工作台是日常使用最频繁的页面。左边看素材，中间处理当前内容，右边观察知识体系。用户不需要来回切换太多页面，就可以完成从输入到查看体系结果的大部分操作。

- 常用操作
  1. 左侧查看原始素材列表。
  2. 中间输入或编辑当前素材。
  3. 右侧查看 AI 体系化知识树。
  4. 点击一键体系化生成新版知识体系。

验收时应同时看页面是否能打开、按钮是否能点击、失败提示是否清楚，以及操作结果有没有真正回到个人知识库数据。

### 6.3 素材界面

素材界面更像原始资料柜。它保留所有输入痕迹，并通过状态、搜索和详情页帮助用户确认哪些素材已经解析，哪些还停留在待处理状态。

- 常用操作
  1. 查看所有素材。
  2. 使用搜索框检索素材。
  3. 按 pending、analyzing、analyzed、failed 筛选。
  4. 打开详情查看 AI 解析结果。

验收时应同时看页面是否能打开、按钮是否能点击、失败提示是否清楚，以及操作结果有没有真正回到个人知识库数据。



## 6.4 知识体系界面

知识体系界面展示的是 AI 整理后的结果。用户在这里看的不是原始碎片，而是主题、分支、节点、来源引用和关联关系。

- 常用操作

1. 展开一级主题和二级分支。
2. 点击知识节点查看内容。
3. 查看来源素材，确认 AI 输出依据。
4. 查看相关节点理解知识关联。

验收时应同时看页面是否能打开、按钮是否能点击、失败提示是否清楚，以及操作结果有没有真正回到个人知识库数据。

## 6.5 问答界面

问答页用于复用知识库。它不是闲聊窗口，理想状态下应该尽量围绕用户已经存入的素材回答。

- 常用操作

1. 输入关于个人知识库的问题。
2. 等待 AI 基于知识上下文回答。
3. 将有价值回答整理为后续素材。

验收时应同时看页面是否能打开、按钮是否能点击、失败提示是否清楚，以及操作结果有没有真正回到个人知识库数据。

## 6.6 设置界面

设置页承担模型、Key、导出和隐私说明等辅助能力。它不在主流程中抢戏，但会影响 AI 能否正常工作，以及用户能不能把数据带走。

- 常用操作

1. 选择 AI 模型。
2. 配置用户自带 API Key。
3. 阅读隐私说明。
4. 导出 JSON 或 Markdown 数据。

验收时应同时看页面是否能打开、按钮是否能点击、失败提示是否清楚，以及操作结果有没有真正回到个人知识库数据。

# 7. 页面功能验证记录

## 7.1 登录页验证

访问路径为 /login。本项主要检查：检查邮箱输入框、密码输入框、登录按钮、注册入口和错误提示。

本次已经对线上登录页完成截图和表单提交。页面能打开，字段能填写，提交后返回 `fetch failed`；该错误被记录为认证链路问题。

- 预期表现：页面或接口按权限返回结果；异常时给出明确提示。

- 证据形式：页面截图、HTTP 状态、命令日志或数据库记录。

## 7.2 工作台验证

访问路径为 /workspace。本项主要检查：检查左侧素材列表、中间素材输入/详情、右侧知识体系树。

页面类验证需要在登录成功后继续补充。当前因为认证链路阻塞，没有伪造这些页面的登录态截图。

- 预期表现：页面或接口按权限返回结果；异常时给出明确提示。
- 证据形式：页面截图、HTTP 状态、命令日志或数据库记录。

## 7.3 素材页验证

访问路径为 /materials。本项主要检查：检查素材列表、搜索框、状态筛选、素材详情和 AI 解析结果。

页面类验证需要在登录成功后继续补充。当前因为认证链路阻塞，没有伪造这些页面的登录态截图。

- 预期表现：页面或接口按权限返回结果；异常时给出明确提示。
- 证据形式：页面截图、HTTP 状态、命令日志或数据库记录。

## 7.4 知识体系页验证

访问路径为 /knowledge。本项主要检查：检查一级主题、二级分支、知识节点、来源素材和相关节点。

页面类验证需要在登录成功后继续补充。当前因为认证链路阻塞，没有伪造这些页面的登录态截图。

- 预期表现：页面或接口按权限返回结果；异常时给出明确提示。
- 证据形式：页面截图、HTTP 状态、命令日志或数据库记录。

## 7.5 问答页验证

访问路径为 /qa。本项主要检查：检查问题输入、AI 回答展示、知识库上下文提示。

页面类验证需要在登录成功后继续补充。当前因为认证链路阻塞，没有伪造这些页面的登录态截图。

- 预期表现：页面或接口按权限返回结果；异常时给出明确提示。
- 证据形式：页面截图、HTTP 状态、命令日志或数据库记录。

## 7.6 设置页验证

访问路径为 /settings。本项主要检查：检查模型选择、API Key 配置、导出按钮和隐私说明。

页面类验证需要在登录成功后继续补充。当前因为认证链路阻塞，没有伪造这些页面的登录态截图。

- 预期表现：页面或接口按权限返回结果；异常时给出明确提示。
- 证据形式：页面截图、HTTP 状态、命令日志或数据库记录。

## 7.7 素材接口验证

访问路径为 `/api/materials`。本项主要检查：检查登录用户素材查询和新增流程。

接口类验证要看三件事：未登录时是否拒绝访问，登录后是否只处理当前用户数据，异常时是否返回可读错误。课程验收时可以用浏览器网络面板或 `curl` 记录请求结果。

- 预期表现：页面或接口按权限返回结果；异常时给出明确提示。
- 证据形式：页面截图、HTTP 状态、命令日志或数据库记录。

## 7.8 解析接口验证

访问路径为 `/api/analyze`。本项主要检查：检查 AI 解析输出字段是否完整。

接口类验证要看三件事：未登录时是否拒绝访问，登录后是否只处理当前用户数据，异常时是否返回可读错误。课程验收时可以用浏览器网络面板或 `curl` 记录请求结果。

- 预期表现：页面或接口按权限返回结果；异常时给出明确提示。
- 证据形式：页面截图、HTTP 状态、命令日志或数据库记录。

## 7.9 体系化接口验证

访问路径为 `/api/systematize`。本项主要检查：检查任务创建、节点写入、版本保存和错误处理。

接口类验证要看三件事：未登录时是否拒绝访问，登录后是否只处理当前用户数据，异常时是否返回可读错误。课程验收时可以用浏览器网络面板或 `curl` 记录请求结果。

- 预期表现：页面或接口按权限返回结果；异常时给出明确提示。
- 证据形式：页面截图、HTTP 状态、命令日志或数据库记录。

## 7.10 问答接口验证

访问路径为 `/api/qa`。本项主要检查：检查服务端上下文读取和 AI 回答返回。

接口类验证要看三件事：未登录时是否拒绝访问，登录后是否只处理当前用户数据，异常时是否返回可读错误。课程验收时可以用浏览器网络面板或 `curl` 记录请求结果。

- 预期表现：页面或接口按权限返回结果；异常时给出明确提示。
- 证据形式：页面截图、HTTP 状态、命令日志或数据库记录。

## 7.11 JSON 导出接口验证

访问路径为 `/api/export`。本项主要检查：检查用户数据导出完整性。

接口类验证要看三件事：未登录时是否拒绝访问，登录后是否只处理当前用户数据，异常时是否返回可读错误。课程验收时可以用浏览器网络面板或 `curl` 记录请求结果。

- 预期表现：页面或接口按权限返回结果；异常时给出明确提示。
- 证据形式：页面截图、HTTP 状态、命令日志或数据库记录。

## 7.12 Markdown 导出接口验证

访问路径为 `/api/export/markdown`。本项主要检查：检查知识体系文本导出格式。

接口类验证要看三件事：未登录时是否拒绝访问，登录后是否只处理当前用户数据，异常时是否返回可读错误。课程验收时可以用浏览器网络面板或 `curl` 记录请求结果。

- 预期表现：页面或接口按权限返回结果；异常时给出明确提示。
- 证据形式：页面截图、HTTP 状态、命令日志或数据库记录。

## 8. GitHub 与 Vercel 部署验证

### 8.1 GitHub 仓库检查

确认远程仓库为 <https://github.com/usedare/ordknow>，当前分支为 `main`，提交内容包含项目源代码、Supabase 迁移、文档生成脚本和最终报告。

- 证据：GitHub 仓库页面截图和 `git push` 记录。
- 通过标准：仓库可访问，构建通过，线上页面可打开，关键配置已说明。

### 8.2 本地构建验证

在推送前执行 `npm run lint` 和 `npm run build`，确认类型检查与生产构建通过。

- 证据：`npm run lint` 与 `npm run build` 的终端输出截图。
- 通过标准：仓库可访问，构建通过，线上页面可打开，关键配置已说明。

### 8.3 推送流程

通过 `git add`、`git commit`、`git push origin main` 将当前工作区推送到 `usedare/ordknow` 仓库。

- 证据：部署过程记录、环境变量说明和后续风险说明。
- 通过标准：仓库可访问，构建通过，线上页面可打开，关键配置已说明。

### 8.4 Vercel 自动部署

Vercel 已绑定项目 <https://ordknow.vercel.app>，GitHub 推送后应触发自动构建或保持当前线上版本可访问。

- 证据：`ordknow.vercel.app` 页面访问截图和 HTTP 检查日志。
- 通过标准：仓库可访问，构建通过，线上页面可打开，关键配置已说明。

### 8.5 线上访问验证

访问 <https://ordknow.vercel.app> 和 `/login`，确认页面能正常响应，并将截图作为测试证据。

- 证据：`ordknow.vercel.app` 页面访问截图和 HTTP 检查日志。
- 通过标准：仓库可访问，构建通过，线上页面可打开，关键配置已说明。

### 8.6 部署风险

若 Vercel 环境变量缺失，登录或 AI 接口可能报错；文档中需记录环境变量和 Supabase 迁移要求。

- 证据：部署过程记录、环境变量说明和后续风险说明。
- 通过标准：仓库可访问，构建通过，线上页面可打开，关键配置已说明。

### 8.7 真实截图证据

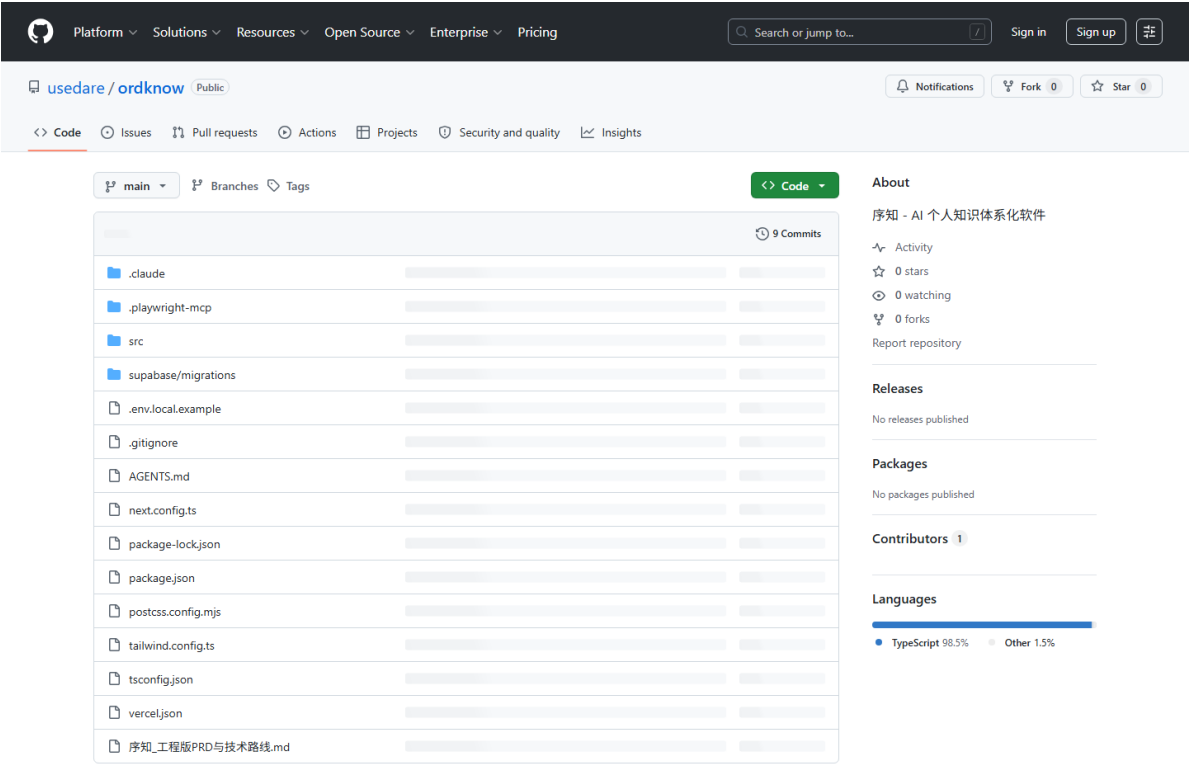


图 21 GitHub 仓库真实页面截图



图 22 Vercel 线上首页真实截图



图 23 Vercel 登录页真实截图



图 24 线上登录页真实截图

序知

输入无序碎片，输出终身知识体系

登录

test@ordknow.com

\*\*\*\*\*

登录

或

注册新账号

邮箱地址

密码 (至少 6 位)

注册

图 25 测试账号填写截图

序知

输入无序碎片，输出终身知识体系

fetch failed

登录

邮箱地址

密码

登录

或

注册新账号

邮箱地址

密码 (至少 6 位)

注册

图 26 测试账号提交后错误截图

72



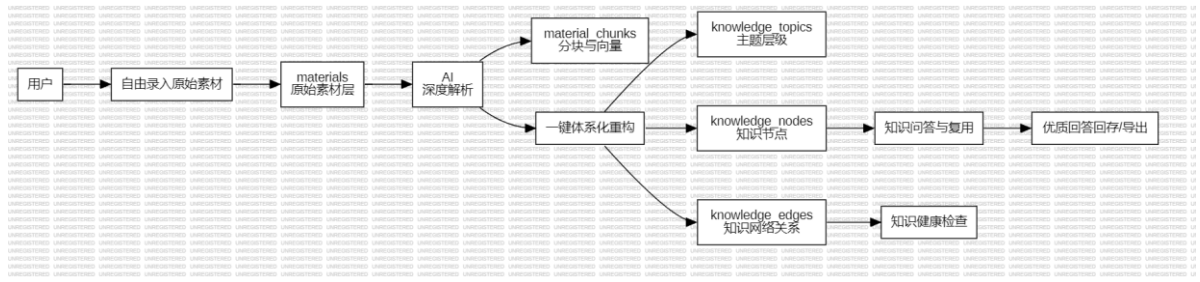


图 27 StarUML 图表真实截图 1

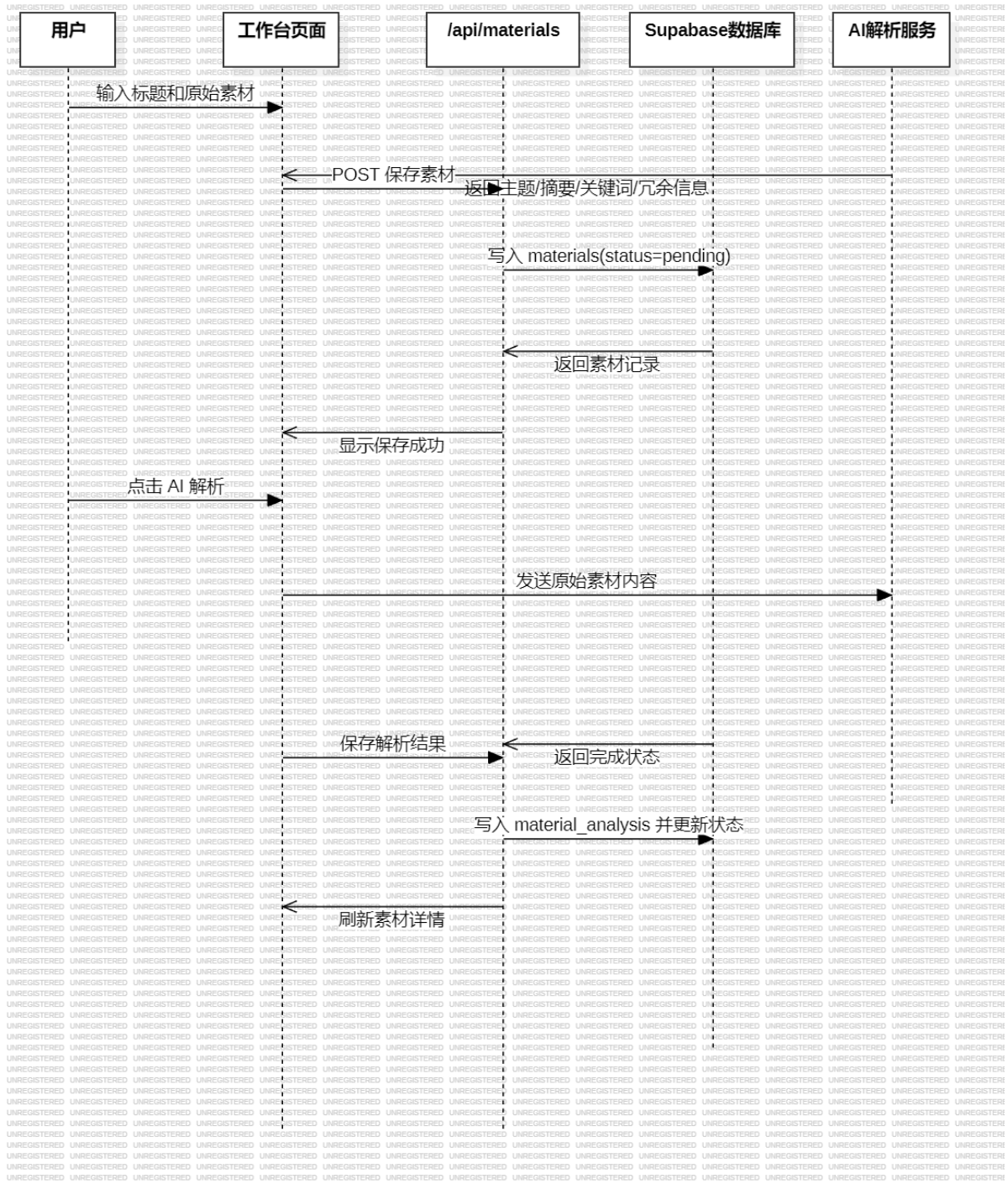


图 28 StarUML 图表真实截图 2

序知 AI 个人体系化知识库软件工程文档

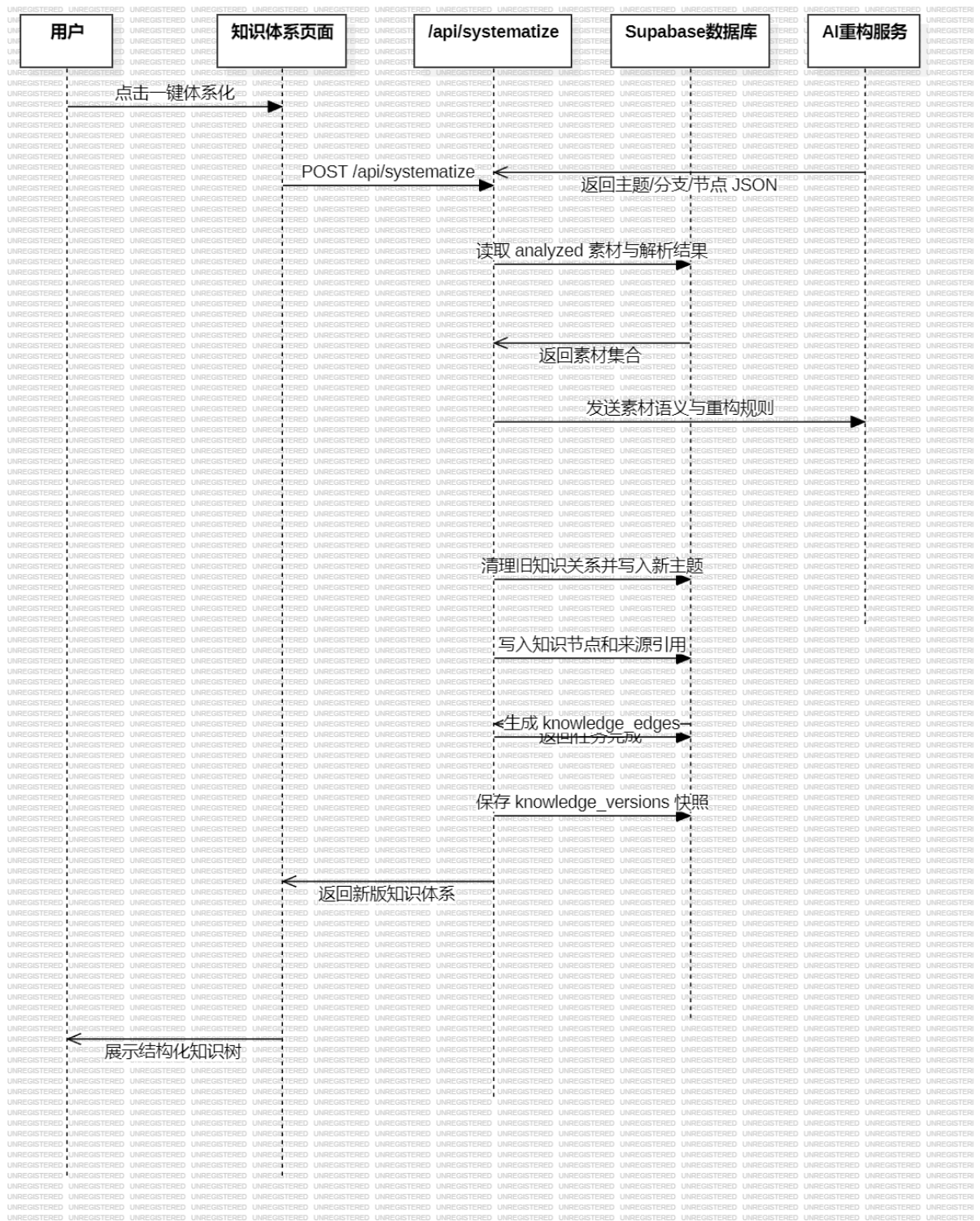


图 29 StarUML 图表真实截图 3

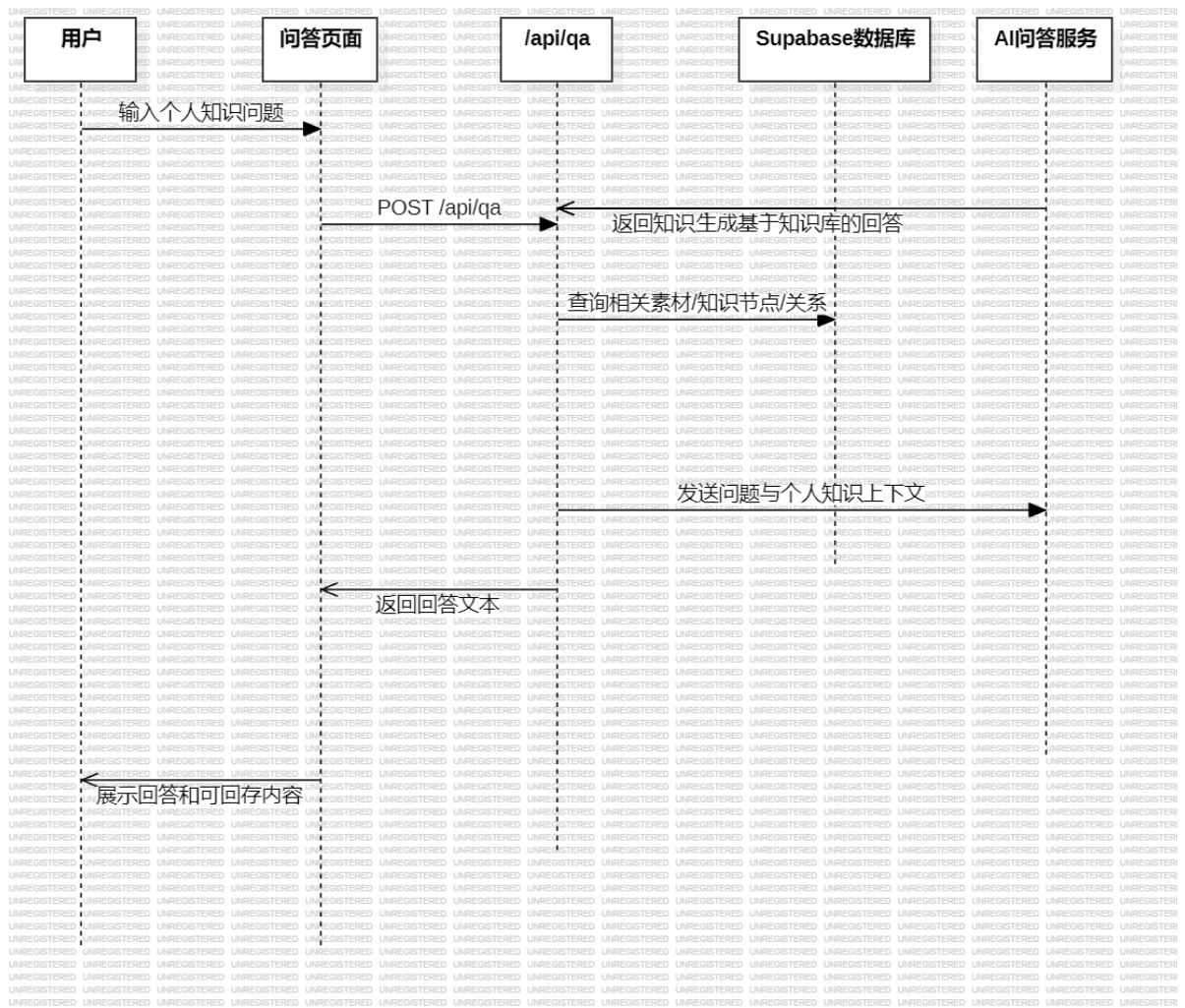


图 30 StarUML 图表真实截图 4

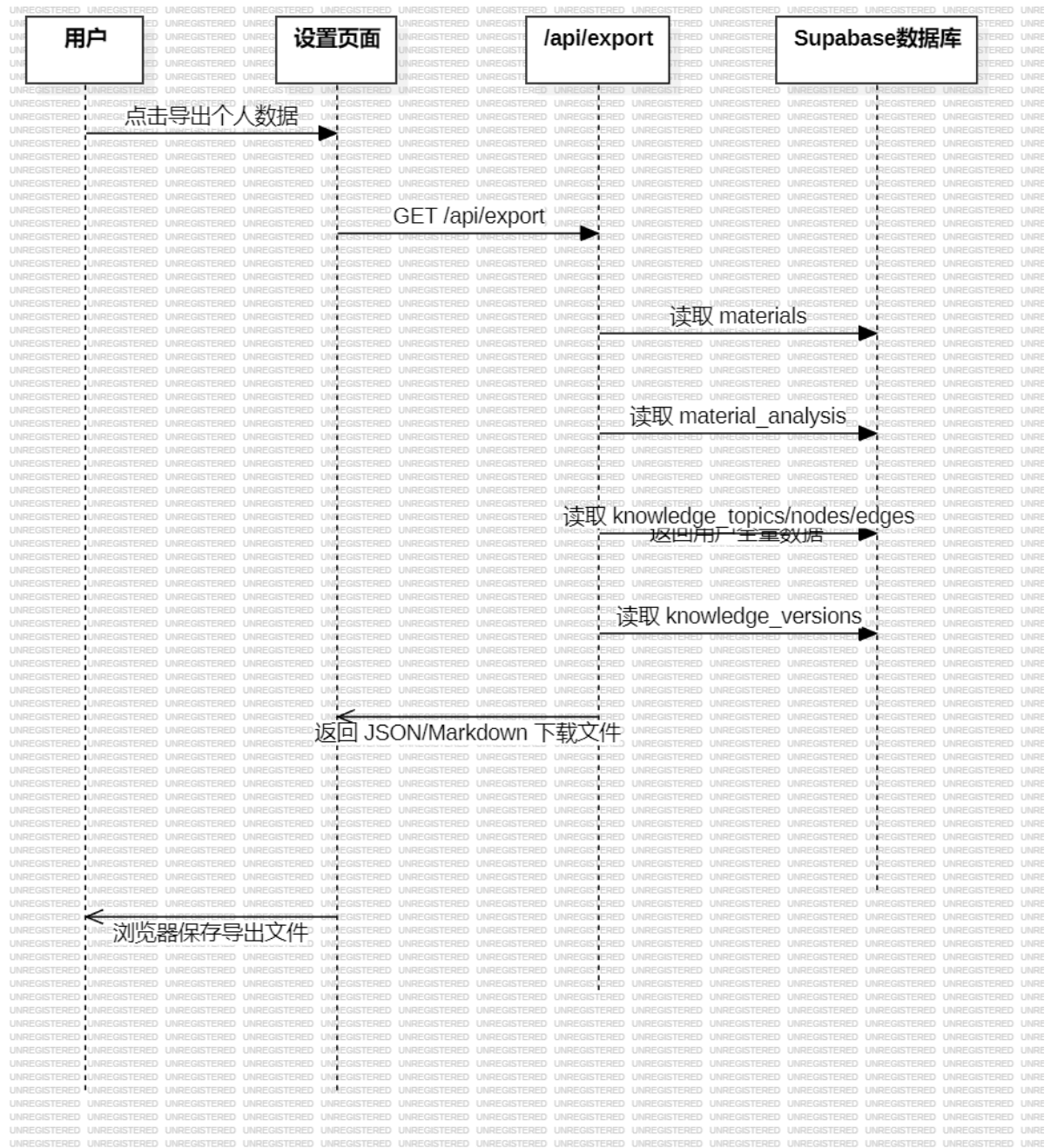


图 31 StarUML 图表真实截图 5

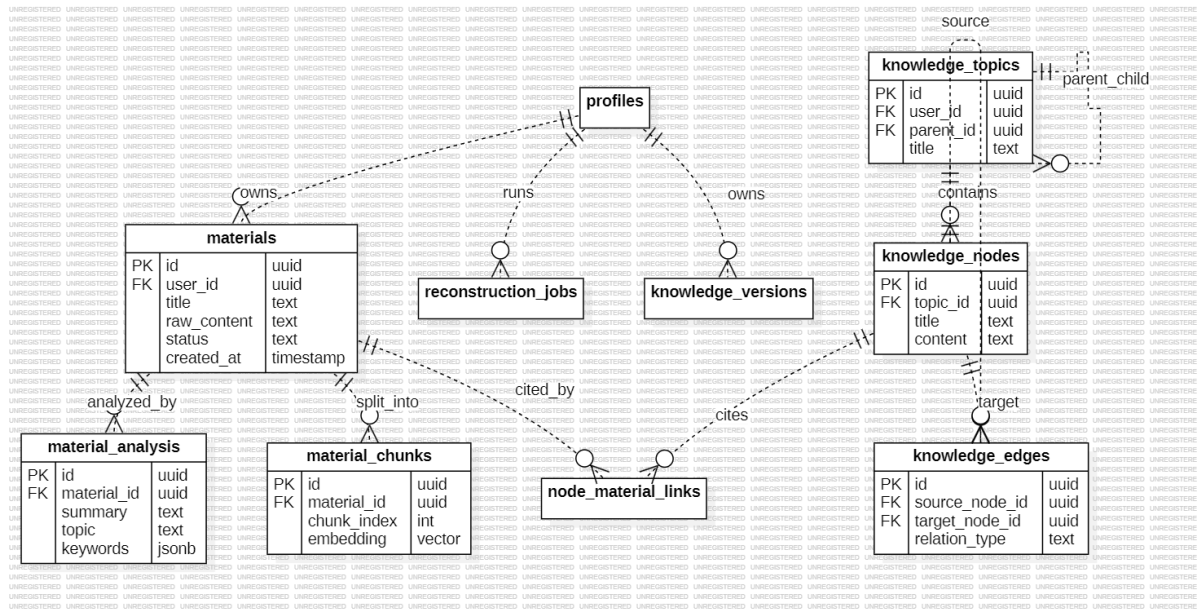


图 32 StarUML 图表真实截图 6

## 9.截图证据索引

表 19 截图证据索引表

| 截图名称              | 来源                       | 证明内容                                        |
|-------------------|--------------------------|---------------------------------------------|
| npm run lint 输出图  | lint.log                 | TypeScript 类型检查通过。                          |
| npm run build 输出图 | build.log                | Next.js 生产构建通过，生成路由表。                       |
| HTTP 检查输出图        | http_checks.log          | 线上入口可访问，未登录访问进入登录保护。                        |
| StarUML 总体流程图     | StarUML 当前项目             | 证明流程图由 StarUML 绘制。                          |
| StarUML 时序图组      | StarUML 当前项目             | 证明核心业务时序图由 StarUML 绘制。                      |
| StarUML ER 图      | StarUML 当前项目             | 证明数据库关系图由 StarUML 绘制。                       |
| GitHub 仓库截图       | usedare/ordknow          | 代码已上传到指定仓库。                                 |
| Vercel 线上截图       | ordknow.vercel.app       | 项目已部署并可访问。                                  |
| 测试账号登录截图          | Chrome CDP 自动化           | 证明 test@ordknow.com 表单提交后线上返回 fetch failed。 |
| 登录阻塞截图            | ordknow.vercel.app/login | 证明当前登录态流程卡在认证服务请求，未伪造后续页面。                  |

## 10.补充验收记录

以下内容是对测试章节的补充。它不再用密集表格列满页面，而是按真实验收顺序说明每一步该看什么、这次实际看到什么、后续怎么补测。

## 10.1 登录与认证链路复测记录

这次拿到测试账号后，第一件事不是直接补工作台截图，而是先验证登录链路。原因很简单：序知所有业务页面都在登录态后面，如果认证没有打通，后面素材、知识、问答的截图都不可信。

实际结果比较明确。登录页可以访问，邮箱和密码字段可以填写，按钮也能提交；提交后页面没有跳到 /workspace，而是在登录页展示 `fetch failed`。这个现象说明前端页面基本存在，阻塞点在服务端认证请求或 Supabase 连通性。

- 复测要点
  - 保留登录前截图，证明线上入口可访问。
  - 保留账号填写截图，证明测试账号确实进入了表单。
  - 保留错误结果截图，证明没有绕过失败点。
  - 复测时先看 Vercel 环境变量，再看 Supabase Auth 请求日志。
  - 登录打通后，第一张补图应是 /workspace 的登录态首页。

本项的判断口径很朴素：能进入工作台才算登录链路通过；只看到登录页或错误提示，只能证明入口存在。当前截图已经能说明问题发生在认证请求阶段，后续修复时不要先改业务页面。

### 记录结论

本项的判断口径很朴素：能进入工作台才算登录链路通过；只看到登录页或错误提示，只能证明入口存在。当前截图已经能说明问题发生在认证请求阶段，后续修复时不要先改业务页面。

## 10.2 原始素材入库复测记录

素材入库是序知的第一步，也是最容易被误解的一步。这里不要求用户整理，也不要求用户把内容写得漂亮。只要内容能作为个人知识库的事实来源，就应该先进入 `materials` 表。

当前代码路径已经具备标题、正文、来源类型和状态字段。前端素材输入框也支持文本、图片、文件、音频和网页入口。因为登录态没有打通，这次没有把测试素材真正写入线上数据库，报告中也没有把它写成已通过。

- 现场记录口径
  - 复测素材标题可使用“软件工程报告真实测试素材”。
  - 素材正文应故意保留几条碎片，符合产品定位。
  - 提交后检查列表是否出现 `pending` 状态。
  - 再刷新页面，确认数据不是只停留在前端状态。
  - 最后打开详情页，确认 `raw_content` 与输入一致。

素材入库复测时不要急着点解析。先确认新增内容是否真的落库，再确认刷新后是否仍在列表中。很多前端问题会在刷新后暴露出来。



### 10.3 AI 单条解析复测记录

解析功能不应该被写成“AI 帮用户润色”。它的职责更窄：读懂单条素材，把核心含义、有效信息、冗余信息、主题、关键词和知识类型提取出来，作为体系化重构的中间层。

这一段复测需要两个前提：素材已经成功入库，AI Provider Key 能被服务端拿到。缺少其中任何一个，解析按钮即使存在，也只能验证到 UI 入口，不能算业务闭环完成。

- 验收时重点看
  - 点击解析前记录素材状态。
  - 点击后观察状态是否进入 `analyzing`。
  - 成功后检查 `material_analysis` 是否有完整字段。
  - 故意清空 Key 时应返回明确错误，而不是页面卡死。
  - 解析失败后原始素材仍应保留。

解析结果最好直接和原文对照。只要解析中出现原文没有的信息，就应记录为 Prompt 约束不足，而不是把它当成模型能力强。

## 10.4 文本分块与向量检索复测记录

分块和向量不是用户每天都会直接看到的功能，但它决定了后续问答和知识关联能不能做深。长素材如果整段塞给模型，成本高，也容易丢重点；切成 `chunk` 后再做向量，检索才有空间。

这部分验收不一定需要复杂界面。更稳的方式是准备一段长文本，触发解析后检查 `material_chunks` 表，看 `chunk_index`、`content`、`embedding` 是否按预期生成。

- 后续补测动作
  - 准备一条超过普通摘要长度的长素材。
  - 解析后检查 `chunks` 数量是否大于 1。
  - 检查 `chunk` 顺序是否稳定。
  - `Embedding` 服务失败时，素材解析不应整体损坏。
  - 后续搜索功能应优先复用这些分块结果。

向量检索的验收可以先从数据库侧做，不必一开始就追求漂亮图谱。只要 `chunk` 和 `embedding` 稳定生成，后续搜索和问答就有继续优化的基础。

## 10.5 一键体系化复测记录

一键体系化是序知最核心的验收点。它不是把素材简单列出来，而是要重新排序、分层、合并重复内容，再写入主题、节点、来源引用和版本快照。

这一步必须等至少几条素材解析成功后再测。只有一条素材也能生成节点，但很难看出“体系化”的价值。更好的测试数据是三到五条同主题但顺序混乱的素材。

- 复测要点
  - 准备几条同类碎片素材，例如 AI 知识库、检索、知识网络。
  - 确认这些素材状态为 `analyzed`。
  - 点击一键体系化后观察任务状态。
  - 检查 `topics`、`nodes`、`node_material_links` 是否同时写入。
  - 确认每个节点能追溯到 `source_material_ids`。

体系化复测不要只看页面有没有树。更重要的是看节点是否有来源、版本是否保存、失败时任务状态是否清楚。

### 记录结论

体系化复测不要只看页面有没有树。更重要的是看节点是否有来源、版本是否保存、失败时任务状态是否清楚。

## 10.6 知识网络与关联关系复测记录

如果只有主题树，序知会接近普通目录软件。知识网络的意义在于，散落在不同时间的素材可以因为共同来源、前置依赖或相互补充而连起来。

当前实现中，`related` 关系主要来自共同来源素材。这个策略不复杂，但足够支撑第一版验收：同一条素材支撑多个节点时，节点之间确实存在天然联系。

- 现场记录口径
  - 找一条同时支撑多个知识节点的素材。
  - 体系化后检查 `knowledge_edges` 是否生成 `related` 边。
  - 在节点详情页查看相关节点。
  - 确认关系只出现在当前用户数据中。
  - 后续可扩展 `prerequisite`、`supports`、`contradicts` 等类型。

知识关系的第一版不必追求复杂，但至少要是能解释关系从哪里来。如果关系来源说不清，后续图谱会变成装饰。

## 10.7 知识问答复测记录

知识问答页的验收重点不是模型回答得多漂亮，而是它有没有使用用户自己的知识库。一个看起来流畅但完全脱离素材的回答，对序知来说反而是不合格的。

登录修复后，可以先提一个简单问题：知识库里目前有哪些主要主题？如果系统能引用已有素材或节点数量，说明上下文组织路径基本打通。

- 验收时重点看
  - 先在知识库准备两到三个主题。
  - 打开问答页输入问题。
  - 观察回答是否出现与素材一致的主题词。
  - 检查 `sources` 数量是否合理。
  - 勾选保存到知识库时，应产生新的回存素材或记录。

问答复测时应准备一个只有知识库能回答的问题。这样能区分系统是在读用户材料，还是只是在给出通用回答。

## 10.8 导出与数据可迁移复测记录

导出功能看起来像附加功能，但对个人知识库很重要。用户把长期知识放进系统后，必须能把数据带走，否则这个产品会让人不放心。

JSON 导出适合备份和迁移，Markdown 导出适合阅读和提交。两个格式的验收重点不同：JSON 看完整性，Markdown 看结构和可读性。

- 后续补测动作
  - JSON 文件应包含素材、解析、主题、节点、关系和版本。
  - Markdown 文件应保留主题层级。
  - 节点内容下方应能看到来源信息。
  - 没有知识体系时，导出也应给出合理空结果。
  - 下载文件名应带日期，便于归档。

导出功能的检查要打开下载文件看内容。只点按钮并不能证明导出成功，更不能证明数据完整。

## 10.9 设置页与用户 Key 复测记录

设置页不应该做得很花，但它必须讲清楚 AI 服务和用户数据的关系。用户 Key 保存在浏览器 localStorage 中，服务端只在请求时临时使用，这是当前实现的边界。

复测时可以切换模型，保存后刷新页面，确认选择仍然存在。再触发一次解析或问答，看请求是否按当前模型配置发送。

- 复测要点
  - 检查 DeepSeek、MiMo、SiliconFlow Key 输入框。
  - 切换模型后点击保存配置。
  - 刷新页面确认配置仍在当前浏览器中。
  - 隐私说明应明确素材会发送到所选 AI 服务。
  - 不要把用户 Key 写入数据库或文档截图。

设置页复测时要避免把真实 API Key 截进报告。截图可以展示输入框和模型选项，但密钥内容必须留空或遮蔽。

## 10.10 异常场景复测记录

异常场景在课程项目里经常被一笔带过，但序知这种 AI 应用不能只测顺风流程。模型失败、JSON 解析失败、数据库写入失败、素材为空、未登录访问都应该有明确行为。

这次已经真实遇到了认证链路失败，所以文档没有把它藏起来。后续继续验收时，也应保持这种口径：失败就是失败，写清楚失败点和下一步处理。

- 现场记录口径
  - 空素材不能提交。
  - AI Key 错误时不应覆盖原始素材。
  - 体系化无素材时应返回可读提示。
  - 未登录 API 请求不能返回业务数据。
  - 网络错误应在页面上给出用户能理解的提示。

异常测试的价值在于暴露边界。当前登录失败就是一个真实边界，应该写进报告，而不是绕过去。

### 记录结论

异常测试的价值在于暴露边界。当前登录失败就是一个真实边界，应该写进报告，而不是绕过去。



### 10.11 页面截图补充计划

当前文档已经包含线上登录页、账号填写、登录失败、GitHub、Vercel、StarUML、lint、build 和 HTTP 检查截图。缺少的是登录成功后的业务页面截图。

这些截图不能靠静态页面或假数据拼出来。登录修复后，应按真实操作补齐工作台、素材页、知识体系页、问答页、设置页，并保留每张图对应的操作说明。

- 验收时重点看
  - 第一张补工作台，证明登录态进入主界面。
  - 第二张补新增素材后状态。
  - 第三张补解析结果或解析失败提示。
  - 第四张补知识树和节点详情。
  - 第五张补问答提交后的回答或错误。

截图补充计划不是装饰清单。每张图都应该对应一个验收结论，否则图片再多也只是堆材料。

## 10.12 当前版本结论

从工程角度看，序知的代码结构、数据库迁移、AI 处理路径、知识网络数据表和部署入口都已经搭起来了。`lint` 和 `build` 能通过，说明项目至少具备继续联调的基础。

从产品验收角度看，当前最大的阻塞是认证链路。它不影响“代码是否存在”的判断，但会影响“真实用户是否能完成闭环”。所以最终结论应写得克制：工程主体完成，线上登录链路待修复，登录后业务闭环需要再跑一次完整验收。

- 后续补测动作
  - 已完成：代码、数据库结构、构建、部署入口、图表和报告主体。
  - 已发现：测试账号登录返回 `fetch failed`。
  - 未伪造：登录态业务页面截图。
  - 下一步：修复 Supabase Auth 连通性。
  - 复测顺序：登录、入库、解析、体系化、问答、导出。

当前版本的结论不宜写得过满。它已经具备工程主体，但线上闭环还被认证链路卡住。这个判断比空泛地写“系统运行良好”更可信。

### 记录结论

当前版本的结论不宜写得过满。它已经具备工程主体，但线上闭环还被认证链路卡住。这个判断比空泛地写“系统运行良好”更可信。